

UAB

Universitat Autònoma de Barcelona

Faculty of Science

Design, Integration, and Validation of a Modular
LLM-Based Interactive Architecture

Juan David Bendek Williamson

Supervised by Raquel Ros

Institution / Lab: IIIA-CSIC / Universitat Autònoma de Barcelona

*Thesis submitted as a requirement for the degree of
MSc in Modelling for Science & Engineering*

Academic year: 2025–2026

Date: July 22, 2026

Compromís d'obra original

L'ESTUDIANT QUE PRESENTA AQUEST TREBALL DECLARA QUE:

1. Aquest treball és original i no està plagiat, en part o totalment
2. Les fonts han estat convenientment citades i referenciades
3. Aquest treball no s'ha presentat prèviament a aquesta Universitat o d'altres

I perquè així consti, afegeix a aquesta plana el seu nom i cognoms i la signatura:

Abstract

Large language models have shown strong potential for instruction understanding and high-level reasoning in human-robot interaction. Their integration into a robot nevertheless requires more than a planner call: dialogue state, symbolic grounding, capability descriptions, execution admission, robot-facing skills, feedback, and user-facing reporting must remain coherent across a distributed runtime. This thesis presents a modular LLM-based interactive architecture and evaluates its implementation on the NAO robot.

The system separates dialogue management, semantic routing, symbolic grounding, task planning, deterministic orchestration, and skill execution. A chatbot interprets each turn and selects a conversational, knowledge, or execution route. A planner generates structured plans over a reviewed skill registry, while an orchestrator admits requests, validates plans, dispatches real or simulated skills, and returns typed execution feedback. A knowledge and perception path provides compact grounded context without allowing planning-time state to stand in for execution evidence.

The contribution is an inspectable interactive architecture and a validation method for its principal boundaries. The validation follows complete user turns together with focused planner, skill-execution, and controlled failure tests. It examines route correctness, plan validity, entity and role preservation, action order, symbolic postconditions, failure handling, and user-facing reporting.

The final evidence package contains 108 case records from 33 retained structured traces. The frozen primary configuration produced 64 passes and eight failures across 72 records. The twenty-two-case end-to-end questionnaire produced twenty passes and two failures. Controlled beginning- and middle-stage skill failures were recovered through versioned replanning, while incomplete capability-extreme plans were rejected before execution. Residual errors were concentrated in model-authored target selection, role preservation, clarification wording, and terminal closure.

Keywords: Human-Robot Interaction, ROS 2, Large Language Models, Task Planning, NAO Robot, Symbolic Grounding, Deterministic Execution, Runtime Validation.

Acknowledgements

Contents

Abstract	i
Acknowledgements	ii
List of Figures	vii
List of Tables	viii
List of Abbreviations	x
1 Introduction	1
1.1 Language-Guided Robot Interaction and Execution	1
1.2 Problem Statement	1
1.3 Research Questions	2
1.4 Objectives	2
1.5 Document Structure	3
2 Background and Literature Review	4
2.1 Human-Robot Interaction	4
2.2 ROS 2 for Distributed Robot Software	4
2.3 Reusable HRI Representations and Interfaces	5
2.4 The NAO Robot Platform	5
2.5 LLMs as High-Level Planners	5
2.6 Embodied Multimodal Models, VLMs, and VLAs	6
2.7 Symbolic State and Knowledge Bases	7
2.8 Execution Monitoring, Replanning, and Clarification	7
2.9 Literature Gap	7

3	System Design	9
3.1	Functional Requirements	9
3.2	Design Principles	9
3.3	High-Level Architecture	10
3.4	Runtime Flow	11
4	Implementation	13
4.1	Runtime Interfaces and Data Contracts	13
4.1.1	Runtime Contract Overview	13
4.1.2	Contract 1: Chatbot Turn Output	14
4.1.3	Contract 2: Planner Request	15
4.1.4	Contract 3: Grounded Context	16
4.1.5	Contract 4: Shared Skill Entry	18
4.1.6	Contract 5: Planner Output	19
4.1.7	Contract 6: Skill Execution Result	22
4.1.8	Contract 7: Execution Feedback	22
4.1.9	Contract 8: Planner Dialogue Act	23
4.1.10	Contract 9: RDF Knowledge State	23
4.2	ROS Components	24
4.2.1	<code>dialogue_manager</code>	26
4.2.2	<code>chatbot_llm</code>	26
4.2.3	<code>planner_llm</code>	27
4.2.4	Goal-Keyed Planning-State Helper	28
4.2.5	Skill Registry (<code>skill_common</code>)	29
4.2.6	<code>nao_orchestrator</code> and Plan Execution	29
4.3	World State and Scene Grounding	30
4.4	Skill Adapters and Controlled Simulation	31
4.5	Models and Runtime Configuration	32
5	Validation Methodology	33
5.1	Validation Scope	33
5.2	Validation Levels	33
5.3	Acceptance Criteria	34

5.4	Full End-to-End Questionnaire	36
5.5	Skill-Execution and Failure Stress Testing	37
5.6	Metrics	38
5.7	Model and Configuration Variability	40
5.8	Research-Question Evidence Map	40
5.9	Threats to Validity	41
6	Results	42
6.1	Frozen Evidence Scope	42
6.2	End-to-End Questionnaire Analysis	43
6.3	Environment and Stateful Knowledge Results	44
6.4	Skill-Execution and Failure-Management Results	44
6.5	Repeated Robustness and Capability Extremes	45
6.6	Model and Backend Evidence	46
6.7	Answers to the Research Questions	48
6.8	Result Summary	48
7	Discussion	50
7.1	What the Final Evidence Demonstrates	50
7.2	Model-Independent Interfaces and Model-Dependent Behaviour	50
7.3	Questionnaire-Level Interpretation	51
7.4	Grounding, Roles, and Symbolic State	51
7.5	Planning, Admission, and Replanning	52
7.6	Application Ablations and Non-Determinism	52
7.7	Controlled Skill Adapters, Physical Claims, and Portability	53
7.8	Implications for the Research Questions	53
8	Limitations and Future Work	54
8.1	Current Limitations	54
8.2	Future Work	55
8.2.1	Model Coupling and Qualification	55
8.2.2	Deterministic Semantic Guards	55
8.2.3	Broader Embodied Validation	55

8.2.4 Evidence and Reproducibility	56
9 Conclusion	57
A Full Runtime Contracts	58
A.1 Runtime Skill Inventory	58
B Launch Commands	61
C Validation Log Template	62
D Final Evaluation Dataset	63
D.1 Dataset Scope and Attachment	63
D.2 Run Inventory	64
D.3 Field Dictionary	64
D.4 Evidence Integrity	65
D.5 Complete Case Index	65
Bibliography	68

List of Figures

3.1	High-level architecture. The language and grounding components provide the planner with the user’s goal and current symbolic context. The planner proposes a skill sequence; the orchestrator validates and coordinates it; and the skills return fresh execution evidence.	10
3.2	Runtime sequence from a user request to an evidence-based response. Dialogue and knowledge turns stop after interpretation; only execution requests continue through planning and skill execution.	12
4.1	Full-stack implementation of the modular interaction architecture. Dialogue interpretation and symbolic grounding provide the planner input; <code>nao_orchestrator</code> admits the request, validates the returned plan, dispatches skill servers, and forwards execution feedback. Robot-facing and simulated adapters share the same typed skill-result boundary.	25
4.2	Internal <code>chatbot_llm</code> path. Dialogue history and grounded context condition a structured semantic decision that either returns a user-facing response or submits an execution request for admission.	27
4.3	Internal <code>planner_llm</code> path, from admitted request through validated plan envelope and feedback-driven goal-state handling.	28
4.4	Execution path implemented by <code>nao_orchestrator</code> . Accepted plans are validated, dispatched in order, and correlated with typed skill results and execution feedback.	30
4.5	Implemented perception and symbolic-grounding pipeline. Reused person-perception components and the local object-grounding path provide bounded scene and knowledge representations to dialogue, planning, and execution skills.	31

List of Tables

3.1	Functional requirements for the proposed interaction and execution architecture.	9
4.1	Runtime contract map and ownership in the active dialogue, planning, and execution stack.	14
4.2	Fields in Contract 1: chatbot turn output.	15
4.3	Fields in Contract 2: minimal planner-facing request.	16
4.4	Fields in Contract 3: grounded context.	18
4.5	Shared skill-entry fields used by planning and execution.	19
4.6	Fields in Contract 5: planner output.	21
4.7	Principal RDF-style predicate families used by the runtime KB.	24
4.8	Planner-visible skills and their implementations.	29
5.1	Validation levels and supported claims.	34
5.2	Validation goals, acceptance rules, and evidence sources.	34
5.3	Twenty-two-case full end-to-end questionnaire.	36
5.4	Validation families and their principal acceptance focus.	37
5.5	Failure position and required recovery evidence.	38
5.6	Validation metrics and their interpretation.	39
5.7	Research questions and validation evidence.	40
6.1	Evaluation evidence by purpose and model scope.	42
6.2	Results for the 22-case end-to-end questionnaire.	43
6.3	Grounding and symbolic-state profiles in the primary frozen evaluation.	44
6.4	Skill-execution and staged failure-management results.	45
6.5	Frozen robustness cases across three repetitions.	46
6.6	Model evidence and qualification boundary.	47

A.1 Compressed runtime skill inventory and registry values. 59

C.1 Validation log template. 62

D.1 Machine-derived archive inventory used by the final dataset. 64

D.2 SHA-256 identifiers for the final supplied evidence files. 65

D.3 Complete index of the 108 retained case records. 66

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
ASR	Automatic Speech Recognition
HRI	Human-Robot Interaction
HTN	Hierarchical Task Network
JSON	JavaScript Object Notation
KB	Knowledge Base
LLM	Large Language Model
NAO	Humanoid robot used as the main experimental platform
PDDL	Planning Domain Definition Language
ROS	Robot Operating System
ROS4HRI	ROS conventions and interfaces for Human-Robot Interaction
TFM	Trabajo de Fin de Máster
TTS	Text-to-Speech
VLA	Vision-Language-Action model
VLM	Vision-Language Model
WME	World Model Enricher

Chapter 1

Introduction

1.1 Language-Guided Robot Interaction and Execution

Human-robot interaction (HRI) requires a robot to connect communication with perception, task reasoning, and embodied action. Automated planning provides one part of this problem. Classical languages such as the Planning Domain Definition Language (PDDL) make actions, preconditions, effects, initial states, and goals explicit (McDermott et al. 1998). Hierarchical task network methods further describe how high-level tasks can be decomposed into executable subtasks (Erol et al. 1994; Nau et al. 2003; Georgievski et al. 2014). Language-model-assisted planning revisits the connection between flexible instructions and grounded action structures (Liu et al. 2023a; Valmeekam et al. 2023; Ahn et al. 2022).

Large language models (LLMs) can transform user utterances into structured intent payloads, infer task goals, and propose multi-step action decompositions. This flexibility is useful because users rarely speak in planner-ready commands. They ask questions, refer to visible people or objects, change goals, and expect responses that remain consistent with the interaction history. LLMs are not inherently grounded in the robot’s live state, available skills, or execution constraints. A linguistically plausible response may select the wrong route, refer to an unavailable entity, propose an unsupported action, or report success without corresponding execution evidence.

This thesis addresses that problem through a modular interactive architecture. The system combines dialogue management, LLM-based semantic interpretation, symbolic grounding, task planning, deterministic orchestration, robot skills, and feedback-driven user communication. The language model participates in dialogue and planning, while explicit interfaces, reviewed skill descriptions, and an orchestrator retain control over execution. The focus is therefore the complete interaction architecture rather than the planner in isolation.

1.2 Problem Statement

Five recurring problems define the scope of this thesis:

- integrations may be tightly coupled to one robot or task domain;
- prompt-only action generation may lack deterministic execution validation;
- dialogue, planning, perception, knowledge, and execution ownership may be ambiguous;
- composite instructions may lose entity roles or ordering as they cross component boundaries;
- clarification, replanning, failure recovery, and evidence-based reporting may be incomplete.

The central problem addressed in this thesis is therefore:

How can a modular robot architecture integrate language-based dialogue, symbolic grounding, task planning, deterministic execution, and feedback while preserving clear responsibility boundaries and evidence-based user communication?

1.3 Research Questions

The work is guided by five research questions. Each question is linked to evidence defined in Chapter 5.

1. **RQ1:** Can the integrated language-guided stack translate natural-language requests into coherent multi-step plans and ordered skill execution for the supported task families?
2. **RQ2:** To what extent does compact symbolic grounding preserve the objects, people, locations, and interaction roles required from dialogue interpretation through execution and reporting?
3. **RQ3:** Does a reviewed skill registry, combined with deterministic admission and validation, reject unsupported skills, malformed arguments, invalid lineage, and robot-specific calls before dispatch?
4. **RQ4:** How does typed execution feedback affect the stack’s ability to recover from failed actions, request clarification, revise a plan, or stop safely when the current plan cannot continue?
5. **RQ5:** Which responsibility and interface boundaries are required to keep language-guided interaction, planning, execution, and user-facing reporting inspectable?

1.4 Objectives

The main objective is to design, integrate, and validate a modular language-model-enabled interactive architecture for a robot. This objective is divided into the following implementation and validation goals:

1. integrate dialogue management, semantic routing, symbolic grounding, task planning, deterministic orchestration, and skill execution through explicit ROS 2 interfaces;

2. define runtime contracts for chatbot output, planner admission, grounded context, skill descriptions, plans, results, execution feedback, dialogue acts, and raw evidence;
3. expose robot capabilities through a reviewed skill registry rather than direct model-generated robot APIs;
4. support conversational, knowledge-query, atomic execution, and composite execution turns without transferring speech or execution ownership to an LLM;
5. constrain candidate plans through deterministic validation before robot actions are dispatched;
6. return typed execution evidence to the planning and dialogue paths so that failure, ambiguity, replanning, and truthful reporting remain observable;
7. validate the integrated stack through full user turns, focused subsystem tests, controlled skill-adaptor outcomes, and symbolic postcondition checks.

1.5 Document Structure

The thesis is organised as follows:

1. Chapter 2 reviews HRI, ROS 2, language-based planning, multimodal models, symbolic grounding, and feedback-driven execution.
2. Chapter 3 presents the system requirements, responsibility boundaries, high-level architecture, and runtime flow as one design chapter.
3. Chapter 4 maps the design to the implemented ROS 2 components, runtime interfaces, skills, and robot adapters.
4. Chapter 5 defines the validation scope, test profiles, questionnaire, metrics, and acceptance rules.
5. Chapter 6 reports the observed trajectory, semantic, failure-injection, KB-effect, model-ablation, and ownership results.
6. Chapter 7 interprets the findings with respect to the research questions and related work.
7. Chapters 8 and 9 present limitations, future work, contributions, and conclusions.

Chapter 2

Background and Literature Review

2.1 Human-Robot Interaction

Human-robot interaction (HRI) studies how people and robots communicate, coordinate tasks, and share physical or social space. In service and social robotics, an interaction cannot be reduced to command recognition. The robot must associate speech and other social signals with people, objects, prior turns, and the state of an embodied task. It must also communicate uncertainty and failure in a manner that is understandable to the user. Surveys of HRI and socially interactive robots therefore describe a multidisciplinary problem spanning robotics, artificial intelligence, perception, dialogue, interaction design, and safety (Fong et al. [2003](#); Goodrich et al. [2007](#)).

These concerns make responsibility boundaries important. Dialogue management must preserve turn and participant context. Perception must retain the provenance and freshness of observations. Planning must operate over capabilities the robot can actually invoke, while execution must reject unsupported or unsafe work. A user-facing response must remain connected to the evidence produced by these layers. The architecture studied in this thesis treats those responsibilities as collaborating components rather than one end-to-end language policy.

2.2 ROS 2 for Distributed Robot Software

The Robot Operating System (ROS) provides the communication infrastructure used to distribute these responsibilities across robot software. Nodes exchange data through typed topics, services, and actions. Topics carry streamed observations and state, services handle bounded request-response operations, and actions support long-running or cancellable work with feedback. ROS 2 extends this model with middleware and deployment mechanisms for distributed robotic systems (Quigley et al. [2009](#); Macenski et al. [2022](#)).

This communication model does not itself define HRI semantics or guarantee correct execution. Its value for this thesis lies in the ability to assign ownership to observable interfaces. Language

models can remain at dialogue and task-planning boundaries, while robot actions, knowledge queries, lifecycle state, and feedback retain typed ROS 2 owners. The same separation supports controlled replacement of real robot skills with deterministic simulated outcomes during validation.

2.3 Reusable HRI Representations and Interfaces

Reusable HRI systems require consistent representations of people and their observable signals. Mohamed et al. (2020) identify the absence of a standard human representation as a barrier to composing perception and interaction pipelines. Their ROS4HRI proposal defines conventions for bodies, faces, voices, groups, and related social signals. This thesis uses those conventions and compatible packages at the public person and speech boundaries; it does not treat ROS4HRI as the planning or execution framework of the system.

In a modular HRI system, information passes through several levels of abstraction. Sensor and speech streams first become tracked observations. Tracking and semantic sources associate those observations with people, objects, and interaction groups. A knowledge representation then exposes bounded entities and relations to dialogue and planning. User intent and grounded roles become candidate skill plans, and admitted plans become typed robot actions whose results feed the knowledge and dialogue paths. Each transition removes detail, so it must preserve the identifiers, roles, freshness, and provenance required by the next layer. An error at one transition can otherwise appear as a prompt, planning, or execution failure downstream.

2.4 The NAO Robot Platform

The physical platform used in this thesis is NAO, a programmable humanoid robot originally developed by Aldebaran Robotics. Gouaillier et al. (2008) describe NAO as an autonomous humanoid platform for research, education, and RoboCup experimentation. Its speech capabilities, articulated body, camera, and established academic use make it suitable for studying spoken interaction, person and object grounding, gaze-oriented actions, motion, and feedback-driven task execution. NAO is the embodied platform in the evaluated system, while higher-level components depend on reviewed skills and ROS 2 interfaces rather than direct model-generated NAOqi calls.

2.5 LLMs as High-Level Planners

Large language models have been studied as high-level planners because they encode procedural and semantic knowledge that can be elicited from natural-language instructions. Huang et al. (2022b) show that language models can decompose embodied tasks into action-like steps, while also exposing the difficulty of mapping unconstrained language to admissible actions. A

linguistically reasonable plan can remain impossible, unsafe, or unsupported by the available robot.

Several approaches ground model proposals in a restricted action set. SayCan combines language-model task knowledge with affordance scores from pretrained skills (Ahn et al. 2022). Code as Policies generates policy code over a supplied API, allowing feedback loops and spatial computations while remaining dependent on reviewed callable primitives (Liang et al. 2022). These approaches motivate the use of an explicit skill registry in this thesis. The model plans over declared capabilities rather than arbitrary natural-language actions.

Reasoning-and-acting methods also show the value of external evidence. ReAct interleaves language reasoning with actions that query tools or environments (Yao et al. 2022). Inner Monologue incorporates perception, success detectors, and human feedback into embodied planning (Huang et al. 2022c). Reflexion stores verbalised feedback as experience across trials without modifying model weights (Shinn et al. 2023). The system in this thesis adopts a narrower runtime loop: a planner emits a structured skill plan, an orchestrator validates and dispatches it, skills return typed results, and subsequent planning or dialogue acts are conditioned on that feedback.

Prompt instructions alone cannot guarantee valid skills, arguments, grounding, or execution order. The implemented stack therefore combines model output with schema validation, registry lookup, argument checks, lineage checks, and deterministic dispatch. The distinction is between using an LLM as one task-reasoning component and allowing it to become the controller of the robot.

2.6 Embodied Multimodal Models, VLMs, and VLAs

Vision-language models (VLMs) connect visual observations with language, while vision-language-action models (VLAs) also map visual and language inputs to robot actions. BLIP-2 connects frozen vision encoders and language models through a lightweight transformer (Li et al. 2023). LLaVA uses visual instruction tuning to support image-conditioned interaction (Liu et al. 2023b). These models show how visual evidence can be transformed into language-compatible representations without themselves defining a complete robot runtime.

Embodied multimodal models extend this direction. PaLM-E incorporates visual and continuous sensor inputs and evaluates them on embodied tasks (Driess et al. 2023). RT-2 represents robot actions as tokens and co-trains on web and robot data (Brohan et al. 2023). OpenVLA provides an open-source VLA trained on a broad collection of robot demonstrations (Kim et al. 2024). This thesis does not train an end-to-end VLA policy. Visual and knowledge evidence instead become compact symbolic context before reaching the dialogue and planning components. Richer spatial or multimodal grounding remains a future extension.

2.7 Symbolic State and Knowledge Bases

High-level robot reasoning requires a representation of the world. Raw perception streams are detailed and transient, while unstructured language summaries may omit identity, freshness, or relation information. A symbolic knowledge base (KB) provides one bridge between these representations. KnowRob represents objects, actions, environments, and task knowledge for robot reasoning (Tenorth et al. 2013; Tenorth et al. 2017). RoboBrain similarly combines multiple knowledge sources to support grounding, perception, and planning (Saxena et al. 2014). Other work studies how interaction can help a robot acquire environmental knowledge and resolve referring expressions (Bastianelli et al. 2013; Grassi et al. 2021).

The implementation follows this principle at bounded scale. Detector-derived observations, person state, and selected KB statements are projected into compact grounded context. The projection retains stable handles, semantic type, interaction role, visibility, and a bounded relation set. Dialogue and planning can use those entities without ingesting unrestricted detector payloads or RDF rows. Skills may still query fresh source evidence at execution time. This separation limits prompt growth and prevents planning-time state from being presented as proof of a later physical effect.

2.8 Execution Monitoring, Replanning, and Clarification

Robot planning cannot stop at plan generation. Actions fail, targets disappear, a user may change the goal, and safe continuation may require clarification. Feedback-oriented language-agent and embodied-planning work supports this closed-loop view (Huang et al. 2022c; Yao et al. 2022; Shinn et al. 2023).

The evaluated stack treats execution feedback as a typed runtime contract. A failed or ambiguous result can stop dispatch, request user input, or return control to the planning component. The orchestrator does not create replacement plans, and the planner does not execute skills. Clarification follows the same ownership path: the planning component emits a structured communication request and the dialogue component realises the user-facing utterance. This keeps recovery observable and avoids inventing missing world facts.

2.9 Literature Gap

Prior work establishes the main ingredients separately. Language models can decompose tasks into action-like sequences (Huang et al. 2022b); affordance and API grounding can restrict proposals to available capabilities (Ahn et al. 2022; Liang et al. 2022); multimodal systems can connect vision, language, and action (Driess et al. 2023; Brohan et al. 2023; Kim et al. 2024); symbolic systems can support robot knowledge and grounding (Tenorth et al. 2013; Saxena et al. 2014); and feedback can condition later reasoning (Huang et al. 2022c; Yao et al. 2022).

The gap addressed here is the integration and validation of these responsibilities within one modular architecture. The contribution combines dialogue lifecycle, semantic route selection, compact symbolic grounding, skill-based planning, deterministic admission and dispatch, typed execution feedback, and evidence-based reporting. It occupies a middle position between fixed intent-to-skill mappings and end-to-end language or VLA policies. The architecture can express composite tasks and react to feedback, while exposing interfaces that permit each transition to be inspected independently.

Chapter 3

System Design

3.1 Functional Requirements

Table 3.1 lists the functional requirements for the complete interaction and execution stack. They cover natural-language ingress, dialogue routing, symbolic grounding, planning, deterministic execution, feedback, and observability. Their implementation owners are introduced in Section 3.3 and detailed in Chapter 4.

ID	Requirement
FR1	The system shall accept natural-language user input through the dialogue stack.
FR2	The system shall distinguish dialogue-only turns from execution-eligible turns.
FR3	The system shall convert execution-eligible user turns into admitted planner requests before invoking the planner.
FR4	The planner shall generate structured skill plans over an explicit skill registry.
FR5	The orchestrator shall validate planner outputs before any robot skill is dispatched.
FR6	The system shall publish execution feedback after relevant plan and step/skill events.
FR7	The planner shall support asking for clarification, replanning and plan failure when execution feedback indicates that the current plan cannot continue safely.
FR8	The system shall expose traces, logs, or runtime messages suitable for validation of the stack.

Table 3.1: Functional requirements for the proposed interaction and execution architecture.

3.2 Design Principles

Four responsibility principles guide the implementation of Table 3.1. They define the component roles before the corresponding ROS 2 package names are introduced.

1. **Dialogue remains distinct from execution.** Greetings, conversation, and questions about the robot’s known state remain non-executable unless the user explicitly requests an action.
2. **The planner proposes structured skill plans.** It selects from the reviewed registry and uses grounded scene context, but does not execute skills, speak directly, or call robot-specific APIs.
3. **The orchestrator remains the execution gate.** It admits execution requests, validates plans, dispatches skills, and publishes typed execution feedback.
4. **Skills own fresh execution evidence.** Perception and execution skills query the knowledge base (KB) or robot state when required. Planning-time context cannot establish that a later action succeeded.

3.3 High-Level Architecture

The design separates dialogue, grounding, planning, orchestration, and robot skills into components with distinct responsibilities. Figure 3.1 presents these components before their individual roles are described.

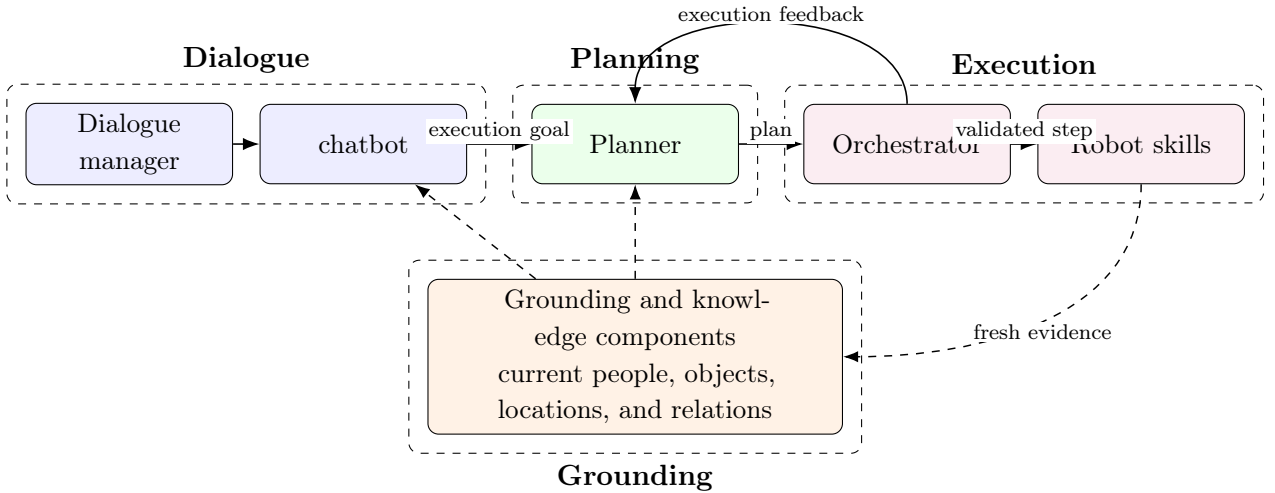


Figure 3.1: High-level architecture. The language and grounding components provide the planner with the user’s goal and current symbolic context. The planner proposes a skill sequence; the orchestrator validates and coordinates it; and the skills return fresh execution evidence.

The main components are:

- The **dialogue manager** maintains the conversation and is responsible for user-facing speech.
- The **language component** interprets each request, distinguishes conversation or knowledge questions from action requests, and extracts an execution goal when needed.
- The **grounding and knowledge components** provide a compact description of

relevant people, objects, locations, and relations without presenting raw sensor data to the language model.

- The **planner** converts an admitted execution goal into an ordered sequence of available robot skills. It proposes actions but does not execute them.
- The **orchestrator** coordinates robot behaviour. It validates the proposed plan, invokes its skills in order, monitors their results, and returns execution feedback.
- The **skills** implement perception, movement, manipulation, motion, and communication capabilities. Each skill returns current evidence about its outcome.

3.4 Runtime Flow

During a normal turn, the dialogue manager receives the user's request and passes it to the language component together with the relevant conversation state. The language component also receives a compact description of the current world. It then selects one of three outcomes: ordinary dialogue, a knowledge answer, or an execution request. Dialogue and knowledge questions return directly to the dialogue manager and do not activate the planner.

For an execution request, the extracted goal and grounded entities are passed to the planner. The planner selects from the available skills and proposes an ordered plan. The orchestrator checks that the plan contains supported skills and valid arguments before invoking anything on the robot.

The orchestrator then executes the accepted steps in order. Each skill returns a result based on current robot or world evidence. These results allow the orchestrator to complete the plan, stop truthfully, or return feedback for replanning or clarification. The dialogue manager communicates the final outcome to the user. Figure 3.2 summarises this flow.

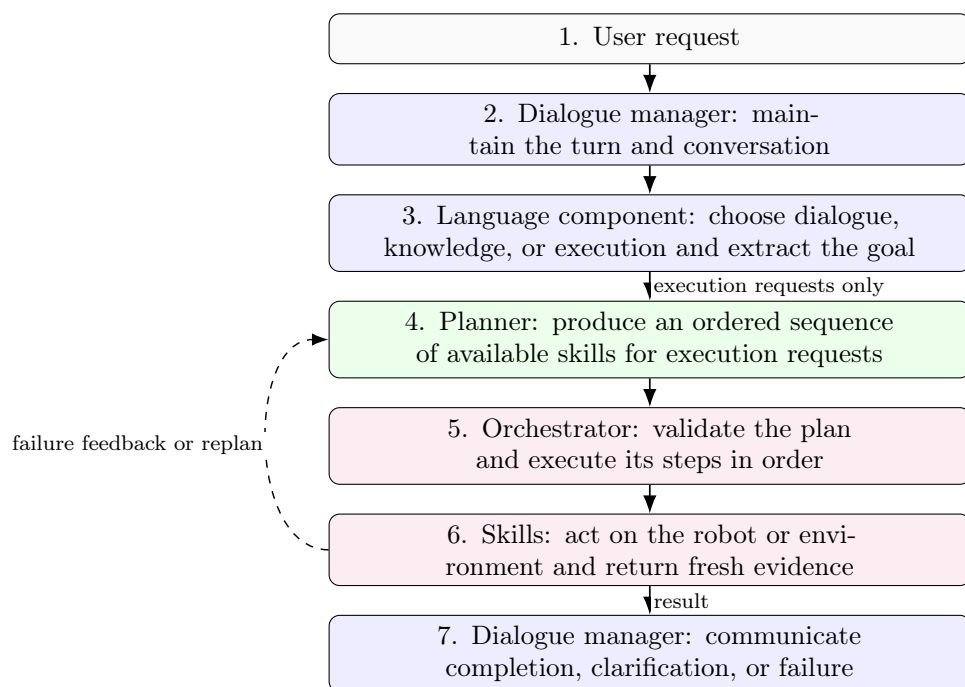


Figure 3.2: Runtime sequence from a user request to an evidence-based response. Dialogue and knowledge turns stop after interpretation; only execution requests continue through planning and skill execution.

Chapter 4

Implementation

This chapter describes how the architecture introduced in Chapter 3 is realised in the ROS 2 workspace. The implementation separates semantic interpretation, planning, deterministic execution, perception grounding, symbolic knowledge, and robot skills. Language-model components interpret requests and propose actions, while typed ROS 2 interfaces and the orchestrator govern execution.

The workspace combines locally developed integration packages with reusable HRI and knowledge components. The implemented contribution is the integration of dialogue management, semantic routing, grounding, planning, orchestration, skills, and feedback into a coherent runtime for the NAO platform.

4.1 Runtime Interfaces and Data Contracts

The implemented components exchange structured messages so that a language-model output cannot become a robot action without validation. Each contract below identifies the data that crosses one implementation boundary and the component that produces, checks, or consumes it.

4.1.1 Runtime Contract Overview

Table 4.1 summarises each runtime contract, its interface or payload, and its owner.

Table 4.1: Runtime contract map and ownership in the active dialogue, planning, and execution stack.

ID	Contract	Interface / payload	Ownership
1	Chatbot turn output	Internal chatbot result before ROS planner publication.	<code>dialogue_manager</code> invokes the chatbot path; <code>chatbot_llm</code> produces the semantic result.
2	Planner request and admission	Candidate topic <code>/nao_orchestrator/planner_request</code> . Admitted topic <code>/planner/request</code> . Planner-facing payload is carried in <code>Intent.data</code> .	<code>chatbot_llm</code> proposes the execution request; <code>nao_orchestrator</code> admits and normalizes it; <code>planner_llm</code> consumes the admitted request.
3	Compact grounding	<code>grounded_context</code> inside Contract 2.	Scene and KB projection provide compact symbolic context for <code>chatbot_llm</code> and <code>planner_llm</code> .
4	Shared skill entry	Registry projection consumed by the planner and orchestrator; its adapter mapping identifies the receiving skill server.	<code>skill_common</code> defines the shared capability shape; <code>planner_llm</code> uses it for prompt construction and <code>nao_orchestrator</code> uses it for plan validation and dispatch.
5	Planner output	Topic <code>/intents</code> . Structured plan payload in <code>Intent.data.plan</code> .	<code>planner_llm</code> produces the plan; <code>nao_orchestrator</code> validates and dispatches executable steps.
6	Skill execution result	ROS action result or service response with a typed skill-result payload.	<code>nao_orchestrator</code> dispatches the call; the skill server owns fresh evidence; verified world-state updates are applied through <code>kb_skills</code> .
7	Execution feedback	Topic <code>/planner/execution_feedback</code> . Structured JSON feedback payload.	<code>nao_orchestrator</code> publishes feedback; a goal-keyed planning-state helper inside <code>planner_llm</code> uses it for replanning, clarification, or failure handling.
8	Planner dialogue act	Topic <code>/planner/dialogue_act</code> . Structured communication request.	<code>planner_llm</code> emits dialogue-act intent; <code>nao_orchestrator</code> relays it; <code>dialogue_manager</code> realizes the final speech.
9	RDF knowledge state and raw evidence	KnowledgeCore statements through <code>/kb/query</code> and <code>/kb/revise</code> ; source evidence includes tracked-person state, <code>/scene/summary</code> , and skill results.	Source components own fact freshness; <code>kb_skills</code> owns KB transport; compact projections and feedback preserve provenance without replacing raw evidence.

The overview keeps only the fields needed to identify each contract. Later sections add the runtime metadata needed for orchestration, feedback-driven planning, tracing, and debugging.

4.1.2 Contract 1: Chatbot Turn Output

The intent in this contract is the user’s goal as extracted by the chatbot. It is not a planner output.

```

1 {
2   "verbal_ack": "Okay, I will do that.",
3   "route": "execution",
4   "confidence": 0.82,
5   "user_intent": {
6     "type": "look_at",
7     "goal_text": "look at the person",

```

```

8   "scene_targets": ["person"],
9   "request_kind": "new_goal"
10  }
11 }

```

Table 4.2 identifies the fields that remain visible after the chatbot completes one semantic routing decision.

Table 4.2: Fields in Contract 1: chatbot turn output.

Field	Meaning
<code>verbal_ack</code>	Optional immediate acknowledgement generated by the chatbot layer. This may be spoken or logged before execution begins, depending on the dialogue policy.
<code>route</code>	Turn classification produced by <code>chatbot_llm</code> . Supported values include <code>dialogue</code> , <code>knowledge_query</code> , and <code>execution</code> . Only <code>execution</code> turns enter the planner path; <code>dialogue</code> and <code>knowledge_query</code> remain non-execution turns but are still useful for tracing.
<code>confidence</code>	Confidence assigned by the chatbot or intent-extraction stage. It is useful for debugging and possible admission policy, but it is not a planner priority score.
<code>user_intent.type</code>	Normalized user-intent label extracted by <code>chatbot_llm</code> , such as <code>look_at</code> , <code>find_object</code> , <code>scan</code> , or <code>navigate_to</code> .
<code>user_intent.goal_text</code>	Natural-language objective passed toward the planner path as the task to satisfy. This field preserves the user's goal in a compact form.
<code>user_intent.scene_targets</code>	Optional target hints extracted from the utterance, such as <code>person</code> , <code>cup</code> , or <code>table</code> . These are hints, not validated grounded entities.
<code>user_intent.request_kind</code>	Proposed goal transition class, such as <code>new_goal</code> , <code>update_goal</code> , <code>answer_to_clarification</code> , or <code>cancel_goal</code> . The planner gate inside <code>nao_orchestrator</code> should normalize and admit this field before publishing <code>/planner/request</code> .

4.1.3 Contract 2: Planner Request

The planner request is the task description generated by the orchestrator from the chatbot output and consumed by the planner.

```

1 {
2   "goal_id": "goal_turn_123",
3   "request_kind": "new_goal",
4   "goal_text": "look at the person and report completion",
5   "normalized_intents": ["look_at"],
6   "grounded_context": {
7     "entities": [
8       {

```

```

9      "id": "anonymous_person_ehfbf",
10     "label": null,
11     "kind": "person",
12     "class": "Human",
13     "visible": true,
14     "relations": []
15   }
16 ]
17 }
18 }

```

Table 4.3 separates request identity, goal semantics, and grounded evidence before the planner is invoked.

Table 4.3: Fields in Contract 2: minimal planner-facing request.

Field	Meaning
<code>goal_id</code>	Identifier for the active objective. It groups replans and feedback for the same task.
<code>request_kind</code>	States whether this is a new goal, an update to the current goal, a cancellation, or another supported transition. <code>chatbot_llm</code> may propose it, but <code>nao_orchestrator</code> normalizes and admits it before publishing <code>/planner/request</code> .
<code>goal_text</code>	Objective extracted from the user utterance. This is the central planning input for <code>planner_llm</code> .
<code>normalized_intents</code>	Intent labels extracted by <code>chatbot_llm</code> . They help <code>planner_llm</code> choose suitable skills from the available skill registry.
<code>grounded_context</code>	Compact scene graph used by the planner as the current symbolic state of the world. It contains validated entities and relations rather than raw detector output.

4.1.4 Contract 3: Grounded Context

The planner receives a compact, JSON-native view of relevant world state. This is best described as a compact scene graph derived from the KB, continually updated by the perception pipeline (human/person detection and object perception). Raw KB triples, detector metadata, and tracker-derived details remain available through scene/KB evidence paths to skills, but they are not duplicated into every LLM prompt.

```
1 {
2   "grounded_context": {
3     "entities": [
4       {
5         "id": "cup_jrjic",
6         "label": "cup",
7         "kind": "object",
8         "class": "Cup",
9         "visible": true,
10        "relations": [
11          {"predicate": "dbp:color", "object": "blue"},
12          {"predicate": "oro:isOn", "object": "table_1"}
13        ]
14      },
15      {
16        "id": "anonymous_person_ehfbf",
17        "label": null,
18        "kind": "person",
19        "class": "Human",
20        "visible": true,
21        "relations": []
22      }
23    ]
24  }
25 }
```

Table 4.4 defines the compact entity and relation fields used by dialogue and planning.

Table 4.4: Fields in Contract 3: grounded context.

Field	Meaning
<code>id</code>	Stable entity handle used by planner steps and skill arguments. This is the preferred reference when a skill needs to target a known person, object, or location.
<code>label</code>	Human-readable name or class hint. It may be empty for anonymous people or entities that should not be assigned a personal name.
<code>kind</code>	Entity category, such as <code>object</code> , <code>person</code> , or another supported runtime category. This prevents people from being treated as generic objects.
<code>class</code>	Compact semantic class used by the planner for reasoning, such as <code>Cup</code> , <code>Human</code> , or <code>Table</code> .
<code>visible</code>	Indicates whether the entity is currently grounded in the latest accepted scene or KB state. It should not be treated as a guarantee that the entity will still be available at execution time.
<code>relations</code>	Bounded list of relevant semantic relations, such as colour, containment, support, proximity, or ownership-style facts. These relations add task-relevant context without exposing the full KB.

4.1.5 Contract 4: Shared Skill Entry

The planner and orchestrator use one shared entry shape to describe a skill before it is placed in a prompt, admitted for execution, or mapped to an action server. The entry separates the information needed to select a skill from the information needed to verify its result.

```

1 {
2   "object_id": "navigate_to",
3   "kind": "skill",
4   "category": "navigation",
5   "aliases": ["go_to", "move_to_location"],
6   "params": ["target", "location"],
7   "required_params": ["target"],
8   "preconditions": ["navigation is enabled",
9                     "target is known or resolvable"],
10  "expected_effects": ["robot arrives at the target location"],
11  "observable_success": ["status"],
12  "failure_modes": ["path_blocked", "unknown_location",
13                  "safety_disabled"],
14  "planner_guidance": ["use for a semantic destination"],
15  "result_schema": {"required": ["skill", "status", "summary_text"]},
16  "safety_flags": ["navigation"],
17  "implementation_status": "fake",
18  "robot_adapter_mapping": "fake_skills.navigate_to",
19  "supports_failure_injection": true,
20  "metadata": {"runtime_callable": true}

```

}

Table 4.5 names the common capability fields consumed by prompt construction, admission, dispatch, and result interpretation.

Table 4.5: Shared skill-entry fields used by planning and execution.

Field	Runtime role
<code>object_id</code> , <code>kind</code> , <code>category</code> , <code>aliases</code>	Canonical planner name, semantic group, and accepted compatibility names.
<code>params</code> , <code>required_params</code>	Typed planner-facing arguments and the subset required before admission.
<code>preconditions</code>	Grounding or state assumptions that must be satisfied or explicitly owned by a scenario. They do not prove a future effect.
<code>expected_effects</code>	Intended state changes or evidence products used to select and order skills.
<code>observable_success</code>	Result or trace evidence that can support completion after execution.
<code>failure_modes</code>	Typed outcomes used by the orchestrator and the goal-state helper to stop, clarify, or request replanning.
<code>planner_guidance</code>	Bounded selection and ordering guidance inserted into the planner prompt.
<code>result_schema</code>	Required and optional result fields, including the summary and evidence payload.
<code>safety_flags</code>	Safety-relevant labels used during admission and dispatch.
<code>metadata.runtime_callable</code>	Whether the registry permits planner-visible runtime use.
<code>implementation_status</code>	Current implementation class, such as first-party, fake, or hybrid.
<code>robot_adapter_mapping</code>	Adapter or skill server that receives the dispatched call.
<code>supports_failure_injection</code>	Whether controlled failure outcomes are available for validation.

The same shape is projected to the planner prompt, deterministic orchestrator checks, controlled fake skill simulation policies, and adapter configuration. Preconditions constrain admission, while expected effects remain hypotheses until a skill returns observable evidence. This distinction is used by `report_result`: a final summary must be derived from result payloads or trace evidence rather than copied from a planner-generated claim. The complete runtime inventory is provided in Appendix A.1.

4.1.6 Contract 5: Planner Output

The planner output is a plan: a sequence of skill steps to achieve the user goal. It should focus on executable content and minimal lineage. Communication policy and retry metadata are still valid runtime fields when used by the stack, but this controlled version foregrounds the ordered `steps` list because that is the essential planner output.

```
1 {  
2   "plan": {  
3     "goal_id": "goal_turn_123",  
4     "plan_id": "plan_123",  
5     "plan_version": 1,  
6     "status": "planned",  
7     "steps": [  
8       {  
9         "id": "step_1",  
10        "type": "skill",  
11        "name": "look_at",  
12        "args": {"target_frame": "anonymous_person_ehfbf"},  
13        "requires": [],  
14        "on_failure": "replan"  
15      }  
16    ]  
17  }  
18 }
```

Table 4.6 identifies the plan and step fields that must survive deterministic validation before dispatch.

Table 4.6: Fields in Contract 5: planner output.

Field	Meaning
<code>goal_id</code>	User objective that this plan attempts to satisfy. It links the planner output to the active goal.
<code>plan_id</code>	Identifier for this concrete plan proposal. It is useful when the runtime keeps plan history or compares revised plans for the same goal.
<code>plan_version</code>	Revision number for replans of the same goal. This helps distinguish the initial plan from later feedback-driven planning outputs.
<code>status</code>	Planning outcome for this plan, such as <code>planned</code> , <code>needs_clarification</code> , or <code>failed_to_plan</code> . This describes whether <code>planner_llm</code> produced an executable plan, not whether the robot has already executed it.
<code>steps</code>	Ordered list of skill calls produced by the planner. This is the essential planner output consumed by <code>nao_orchestrator</code> .
<code>steps[].id</code>	Stable step identifier used for execution feedback, dependency tracking, and possible replan joins.
<code>steps[].name</code>	Skill name selected from the allowed skill registry. The planner should not invent skills or use robot-specific APIs directly.
<code>steps[].args</code>	Skill arguments. They should refer to known entity IDs, explicit primitive values, or arguments supported by the selected skill schema.
<code>steps[].requires</code>	Dependencies between steps, if any. This allows the runtime to know which steps depend on previous successful outcomes.
<code>steps[].on_failure</code>	Recovery policy proposed by <code>planner_llm</code> for this step. If the step fails, <code>nao_orchestrator</code> may fail safely, continue if allowed, request user input, or send feedback back to <code>planner_llm</code> for replanning. Allowed values include <code>replan</code> , <code>clarify</code> , <code>ask_user</code> , <code>continue</code> , <code>ignore</code> , and <code>fail</code> .

Runtime recovery metadata. The current contract supports recovery metadata as skill failure may require controlled recovery. Each step may carry a `retry_budget`, and the plan envelope may also carry a plan-level `retry_budget`. This does not mean that `retry` is a separate `on_failure` value. Instead, retry behaviour is controlled by `nao_orchestrator` using the available retry budget together with the step’s `on_failure` policy. The plan envelope may also carry `communication_policy`. This controls when the planner/dialogue path should communicate progress, completion, clarification needs, or failure.

When a step fails with `on_failure = replan`, `nao_orchestrator` halts or blocks continuation, publishes execution feedback to `planner_llm`, and waits for a revised validated plan before dispatching further actions. This keeps execution authority inside `nao_orchestrator`, while allowing recovery to be driven by feedback-driven planning rather than by hard-coded local replanning.

4.1.7 Contract 6: Skill Execution Result

Every dispatched skill returns a typed result before the orchestrator can mark its step as completed. The result distinguishes the execution status from the evidence that supports it. For state-changing skills, the evidence may also describe world-state changes that must be reflected in the KB. These changes are reported by the skill that observed or produced them; they are not inferred from the planner's expected-effects text in Contract 4.

```
1 {  
2   "skill": "place_object",  
3   "status": "succeeded",  
4   "summary_text": "I placed cup_1 on table_1.",  
5   "evidence": {  
6     "object_id": "cup_1",  
7     "destination_id": "table_1",  
8     "kb_effects": [  
9       {"operation": "remove", "statement": "robot oro:holds cup_1"},  
10      {"operation": "add", "statement": "cup_1 oro:isOn table_1"}  
11    ]  
12  },  
13  "failure": null,  
14  "metadata": {"fake": true}  
15 }
```

The implementation retains the field name `evidence.kb_effects`, but the field describes observed world-state changes that must be reflected in the KB and the orchestrator applies these statements only after a successful result.

4.1.8 Contract 7: Execution Feedback

Execution feedback tells the planner which goal and step were affected, whether the step succeeded or failed, and why. This compact information is sufficient to decide whether to continue, replan, request clarification, or report failure.

```

1 {
2   "goal_id": "goal_turn_123",
3   "status": "failed",
4   "step": {
5     "id": "step_1",
6     "name": "look_at"
7   },
8   "reason": "target frame unavailable",
9   "result_payload": {
10    "target_found": false,
11    "target_kind": "person"
12  }
13 }

```

Runtime feedback may include additional fields used by `nao_orchestrator`, logging, and the goal-state helper. Lineage uses `plan_id` and `plan_version`; event handling may add `event_type`, `validation_status`, `blocking`, and `needs_user_input`; observability may add `timestamp_sec`, `result_summary`, and retry metadata. These fields support execution control and debugging, while the minimal planner-facing feedback remains the goal, step, status, reason, and result payload.

4.1.9 Contract 8: Planner Dialogue Act

Planner dialogue acts represent planner intent to communicate, not direct speech execution. They are relayed through the orchestrator and realized by the dialogue layer.

```

1 {
2   "goal_id": "goal_turn_123",
3   "act": "ask_clarification",
4   "await_user_response": true,
5   "reason": "multiple visible people",
6   "text_hint": "Which person should I look at?",
7   "slots_needed": ["target_person"]
8 }

```

Allowed dialogue acts are acknowledgement, progress update, clarification, help request, failure explanation, completion notification, and cancellation notification. The planner does not speak directly; it emits a structured communication request.

4.1.10 Contract 9: RDF Knowledge State

The implementation reuses the open-source KnowledgeCore package rather than introducing a new knowledge-base system (Lemaignan et al. 2026). KnowledgeCore represents symbolic world state as an RDF-style graph. Each statement has a subject, predicate, and object. Stable

subjects identify people, objects, places, supports, and the robot; predicates express class membership, attributes, visibility, containment, and spatial relations. The same statement shape is used by preloaded fixtures, detector-derived object facts, explicit user-authorised KB mutations, and verified post-skill effects.

```

1 codex_lab_cup rdf:type Cup
2 codex_lab_cup dbp:color red
3 codex_lab_cup oro:isOn codex_lab_table
4 codex_lab_room oro:contains codex_lab_table
5 person_alex rdf:type Human
6 myself sees person_alex

```

The principal predicate families are shown in Table 4.7. A class statement such as `rdf:type Human` also preserves the semantic distinction between a person and an object even when both can be action targets.

Table 4.7: Principal RDF-style predicate families used by the runtime KB.

Role	Representative predicates	Runtime meaning
Entity class	<code>rdf:type</code>	Distinguishes people, objects, supports, rooms, and other planner-relevant classes.
Attributes	<code>dbp:name</code> , <code>dbp:color</code>	Provides stable semantic labels and disambiguating properties.
Observation	<code>sees</code> , <code>visual-centre</code> , <code>score</code> , <code>source</code> , and <code>last-seen</code> predicates	Records who observed an entity and the bounded provenance/freshness of a detector observation.
Spatial relations	<code>oro:isOn</code> , <code>oro:isAt</code> , <code>oro:isIn</code> , <code>oro:contains</code>	Represents support, destination, room membership, and containment used by grounding and skill effects.

KB mutations use `add`, `revise`, and `remove` operations on this shared statement form through `/kb/revise`; reads use query patterns through `/kb/query`. Transient perception facts carry a finite lifespan, while explicit fixture and verified skill-effect facts follow the lifetime declared by their owner. Reused person-perception components publish person identities and tracked state through their public interfaces (Mohamed et al. 2020). Object detections are handled separately by `nao_scene_grounding`, which publishes `/scene/summary` and detector-derived KB statements.

4.2 ROS Components

Figure 4.1 locates the principal ROS 2 components and the information exchanged between them. It uses one representation of `nao_orchestrator`, whose internal responsibilities include planner-request admission and deterministic plan execution, and focuses on the operational path exercised by the implementation.

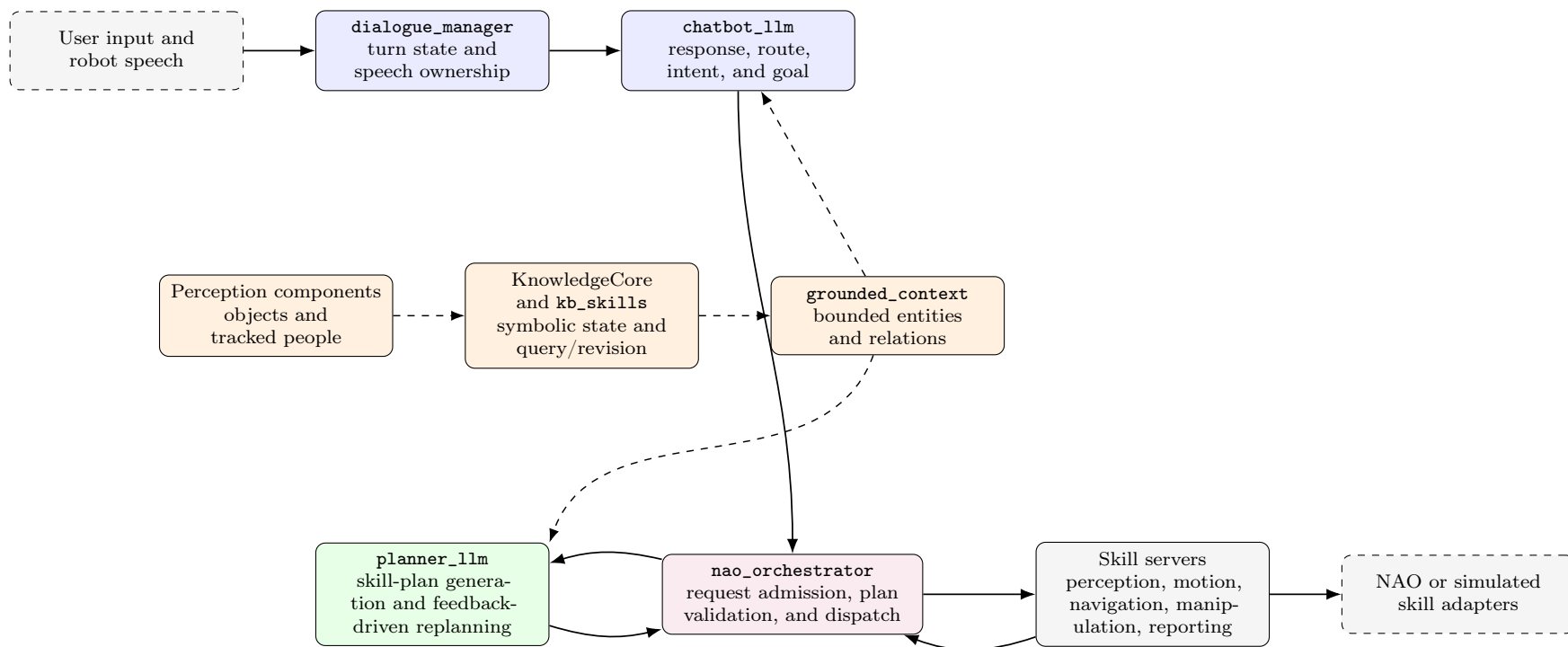


Figure 4.1: Full-stack implementation of the modular interaction architecture. Dialogue interpretation and symbolic grounding provide the planner input; `nao_orchestrator` admits the request, validates the returned plan, dispatches skill servers, and forwards execution feedback. Robot-facing and simulated adapters share the same typed skill-result boundary.

4.2.1 `dialogue_manager`

`dialogue_manager` is the ROS 2 lifecycle component responsible for multimodal communication between the robot and its interlocutors. Following the upstream package design¹, it preserves chronological conversation histories for active interlocutors and associates incoming `hri_msgs/LiveSpeech` streams with the relevant dialogue state. The node invokes the chatbot through `DialogueInteraction`, exposes the high-level `/skill/chat`, `/skill/ask`, and `/skill/say` interfaces, and realises final user-facing expressions through the configured Say sub-skill.

This ownership boundary is also a runtime constraint. Planning and execution components publish structured communication requests through the designated dialogue interfaces, while `dialogue_manager` retains responsibility for the final utterance and its delivery through the speech backend.

Planner dialogue acts return through the same boundary. `planner_llm` publishes structured acknowledgement, clarification, failure, or completion acts on `/planner/dialogue_act`; `nao_orchestrator` relays the admitted act on `/nao_orchestrator/planner_dialogue_act`; and the dialogue layer selects the final wording and speech mechanism. This arrangement provides one user-facing speech authority for each semantic event and makes duplicate acknowledgements or completion reports observable during validation.

4.2.2 `chatbot_llm`

`chatbot_llm` is a lifecycle-managed semantic-routing node built on the existing chatbot scaffold. Its turn callback receives a `DialogueInteraction` request and the conversation history assembled by `dialogue_manager`. Before response generation or planner handoff, it queries the KB through `kb_skills`, reads the most recent `/scene/summary`, and passes both sources through the `PlannerHandoff` projection helper. The resulting `grounded_context`, defined by Contract 3, contains a bounded set of entities, locations, and task-relevant relations instead of raw detector streams or unrestricted KB rows. This projection informs dialogue and knowledge answers on every user turn and is attached to Contract 2 when the selected route produces a planner request.

The turn engine implements the route field from Contract 1 as follows:

- A `dialogue` route covers conversational and task-discussion turns and does not publish planner work.
- A `knowledge_query` route answers from current symbolic evidence without causing a physical action. For example, “what can you see?” can use the current grounded context, whereas “scan and tell me what you see” requests fresh sensing and is eligible for execution.
- An `execution` route constructs planner-facing metadata such as goal text, request kind, normalised intent hints, and target KB entities. It publishes a candidate request to

¹Package source: https://github.com/ieverythng/dialogue_manager.

`/nao_orchestrator/planner_request` without constructing the skill sequence itself.

The chatbot prompt defines the component role, the permitted routes, the response requirements, and the structured intent fields. Runtime context adds the dialogue history, available skill summary, and grounded entities. In the evaluated `response_first` configuration, the first model call produces the acknowledgement and route decision; the intent stage then extracts goal and target metadata under that route. The `intent_first` ablation reverses the stage order and constrains response generation with the previously selected route. Both configurations produce the same Contract 1 output and planner-request format, which allows route consistency to be compared without changing downstream interfaces.

Figure 4.2 summarises the implemented turn path and the two possible outputs: a response returned to the dialogue manager or an execution request submitted for planner admission.

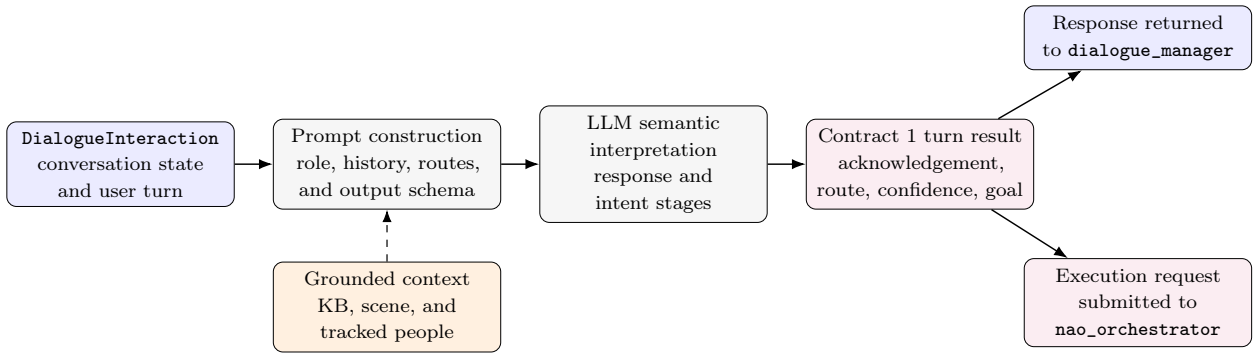


Figure 4.2: Internal `chatbot_llm` path. Dialogue history and grounded context condition a structured semantic decision that either returns a user-facing response or submits an execution request for admission.

4.2.3 `planner_llm`

`planner_llm` is the high-level planning node and contains a goal-keyed planning-state helper for feedback-driven state transitions. The node subscribes to admitted `/planner/request` messages instead of directly consuming chatbot output. Through `planner_common`, each request is normalised into the canonical contract with goal metadata nested under `plan`. The planner prompt combines the goal text, request kind, normalised intent hints, grounded context, the shared skill-entry projection from Contract 4, permitted step types, and the required JSON output envelope.

The planner prompt is assembled from the admitted request, bounded grounded entities, the skill-registry fields relevant to selection and ordering, the permitted step types, and the planner output contract. It specifies the available skills, required arguments, expected effects, observable completion evidence, and failure modes that may require clarification or replanning. This registry projection links the model’s language-level proposal to the typed ROS 2 execution surface.

The model response remains a candidate until the planner engine extracts the JSON object, normalises its schema, validates the required metadata and step fields, and applies one

bounded repair retry when configured. Unsupported skill names, malformed arguments, invalid recovery policies, or unparseable output become typed failure or clarification outcomes. Valid plans are published on `/ intents` for independent orchestrator validation.

Evidence-producing plans receive an additional reporting check. When `report_result` follows a sensing, navigation, motion, or manipulation skill, `planner_common` requires the reporting step to omit planner-authored result wording. Compatibility normalisation removes fields such as `summary_text`, `result_summary`, `text`, `message`, and `utterance` before validation. The reporting step is later resolved from the preceding Contract 6 result payload, preventing an expected effect from being presented as an observed result.

The node also subscribes to `/planner/execution_feedback`. The embedded planning-state helper associates relevant events with the active goal and invokes the planning path when the orchestrator reports completion, failure, a recoverable condition, or a need for user input. The response may be a revised plan or a structured dialogue act. Robot control remains with the orchestrator, while user-facing communication follows the planner-dialogue relay and dialogue manager.

Figure 4.3 shows the planner model call together with its validation, retry, feedback, and publication paths.

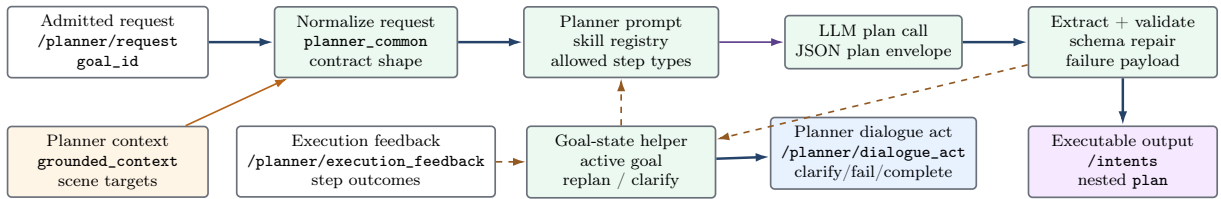


Figure 4.3: Internal `planner_llm` path, from admitted request through validated plan envelope and feedback-driven goal-state handling.

4.2.4 Goal-Keyed Planning-State Helper

The goal-keyed planning-state helper maintains planning state under an active `goal_id`. Each accepted plan carries a `plan_id` and monotonically meaningful `plan_version`; stable `step_id` values allow a newer plan version to preserve completed work when replanning continues the same goal. Feedback from an older or unrelated version is rejected before it can alter the current goal state.

The helper distinguishes planning, replanning, waiting-for-user, completed, and failed states. Step feedback may advance the goal, expose a recoverable condition, or produce a structured dialogue act. If a replacement plan is required, `planner_llm` creates and validates it under a newer version. Execution supervision and step dispatch remain within `nao_orchestrator`; the helper maintains planning order and lineage across these transitions.

4.2.5 Skill Registry (skill_common)

The planner-facing capability model is derived from the canonical skill registry maintained by `skill_common`. Contract 4 and Table 4.5 define the shared entry shape: canonical name and aliases, typed parameters, preconditions, expected effects, observable success evidence, failure modes, result schema, safety flags, runtime availability, and executor mapping. The planner receives a bounded projection of these fields when it selects and orders skills. The orchestrator uses the same names, argument requirements, and availability information when it validates Contract 5 plans and dispatches Contract 6 calls. This arrangement provides a machine-checkable guard against divergence between the prompt-visible capability model and the execution surface.

Table 4.8 lists the planner-visible skill set. Knowledge-base add, revise, and remove operations remain knowledge-service operations and are not included in this registry. Appendix A.1 gives the arguments, expected evidence, failure modes, and executor for each entry.

Table 4.8: Planner-visible skills and their implementations.

Skill	Purpose	Implementation	Controlled simulation
<code>scan</code>	Refresh scene evidence and return a typed scan payload.	First-party <code>/skill/scan</code> server.	Shared scene-fixture control.
<code>find_object</code>	Find one specified object and return whether it was found.	Perception or simulated adapter.	Yes.
<code>inspect_area</code>	Return the objects and people observed in a specified area.	Perception or simulated adapter.	Yes.
<code>navigate_to</code>	Reach a semantic destination such as a person, object, or location.	Navigation or simulated adapter.	Yes.
<code>walk_to</code>	Execute a short body-relative locomotion segment.	Robot movement or simulated adapter.	Yes.
<code>pick_object</code>	Grasp one specified object and return the observed result.	Manipulation or simulated adapter.	Yes.
<code>place_object</code>	Place a held object at a specified destination.	Manipulation or simulated adapter.	Yes.
<code>bring_object</code>	Deliver one specified object to a person or location.	Composite manipulation or simulated adapter.	Yes.
<code>perform_motion</code>	Execute a named body motion, including a greeting wave.	Motion replay or simulated adapter.	Yes.
<code>look_at</code>	Direct the robot’s gaze towards a specified target.	Head, gaze, or simulated adapter.	Yes.
<code>ask_user</code>	Request information needed to continue planning.	Dialogue interface.	Trace-based.
<code>report_result</code>	Report an outcome derived from completed skill results.	Dialogue and reporting interface.	Trace-based.

4.2.6 nao_orchestrator and Plan Execution

`nao_orchestrator` implements deterministic execution of accepted plans. Before dispatch, it validates the plan envelope, ordered steps, registered skill names, arguments, dependencies, failure policies, and goal/plan lineage against the shared skill registry and runtime contracts.

A plan is accepted only when every executable step can be resolved to an available skill interface.

Accepted plans are executed in order through the ROS action or skill-server interface mapped by each registry entry. The orchestrator retains the active goal, plan version, current step, completed results, and pending work. For each step it publishes the corresponding started, succeeded, or failed event on `/planner/execution_feedback`, preserving the identifiers required to associate execution evidence with the correct goal and plan version.

Each dispatched skill must return a typed Contract 6 result before the step can complete. Successful state-changing results may include world-state updates, which are applied through `kb_skills` and verified through a subsequent query when available. On failure, the orchestrator follows the validated step policy: it may stop the plan, continue only when permitted, or publish feedback that requests replanning or clarification. Replacement plans remain the responsibility of `planner_llm`.

Figure 4.4 summarises the validated plan-execution path described above.

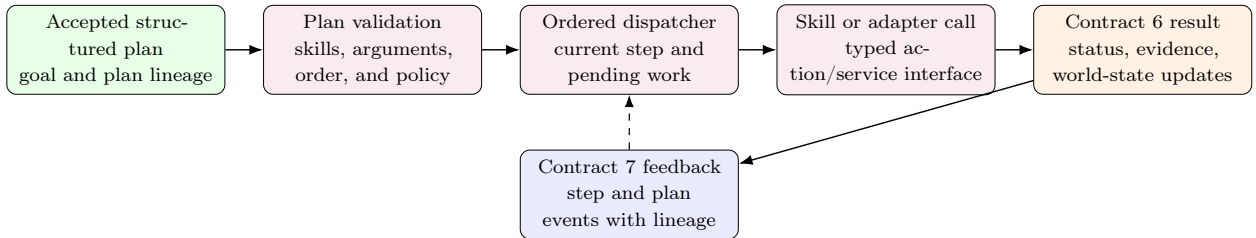


Figure 4.4: Execution path implemented by `nao_orchestrator`. Accepted plans are validated, dispatched in order, and correlated with typed skill results and execution feedback.

4.3 World State and Scene Grounding

The perception path has parallel people and object branches. The person branch reuses `hri_face_detect_yunet`, `hri_person_manager`, and the ROS4HRI person interfaces to provide identities, tracked-person topics, and frame-qualified state (Mohamed et al. 2020). The object branch uses `nao_scene_grounding` to convert detector output into a provider-independent scene representation. A custom YOLO detector was trained for a restricted proof-of-concept object set and integrated with the NAO camera stream through ROS. Its strongest demonstration classes are blueberry, corn, pear, tomato, and zucchini. Detector backends are normalised into one internal observation shape so that downstream components do not depend on the detector message type.

The grounding node filters object detections by confidence and an explicit label allowlist, maps labels to KB classes, and maintains local identifiers using tracker IDs or bounded image-plane matching. Without a tracker ID, the default matcher requires the same label, KB class, and detector source, a displacement of at most 64 pixels, and an observation age of at most two seconds. It publishes `/scene/summary` and writes accepted observations as transient KnowledgeCore statements through `/kb/revise`. Under the default configuration,

KB observations have a four-second lifespan and are refreshed no more often than once per second; a local object is removed from the current scene summary after 4.5 seconds without a new detection. An object marked visible in `grounded_context` is therefore present in this bounded current-scene window. People remain managed by the tracked-person pipeline and are merged when compact symbolic context is assembled.

`kb_skills` provides the query and revision interface for KnowledgeCore. The chatbot formats a bounded query result as an internal knowledge snapshot, combines it with the current scene summary and tracked-person state, and emits the Contract 3 `grounded_context`. The projection retains entities, locations, and bounded relations while omitting unrestricted raw rows and detector payloads.

Planning context and execution evidence consequently have different roles. Current symbolic state supports knowledge answers and plan selection. Execution claims require a fresh Contract 6 result from the responsible skill. When `pick_object`, `place_object`, or `bring_object` changes the world state, the result identifies the corresponding removals and additions; the orchestrator reflects them through `kb_skills` and verifies the required postconditions when the query interface is available.

Figure 4.5 presents the implemented object, person, knowledge, and compact-grounding paths.

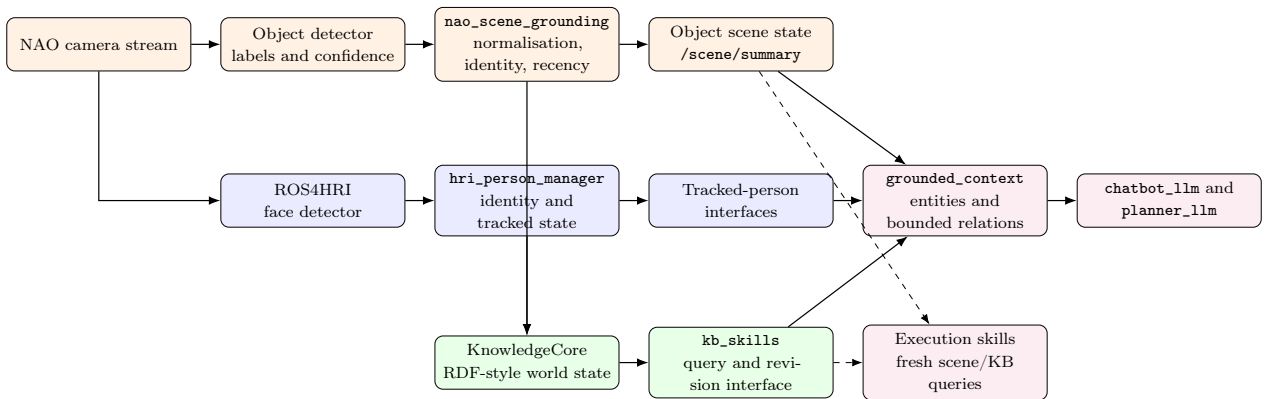


Figure 4.5: Implemented perception and symbolic-grounding pipeline. Reused person-perception components and the local object-grounding path provide bounded scene and knowledge representations to dialogue, planning, and execution skills.

4.4 Skill Adapters and Controlled Simulation

The implementation exposes the planner-visible capabilities in Table 4.8 through a common adapter boundary. Robot-facing adapters invoke the available physical implementation, whereas the package named `fake_skills` supplies deterministic simulated action servers for controlled software evaluation. Both paths preserve the Contract 6 result shape. Simulated results additionally record `metadata.fake=true`, the scenario and result mode, the selected policy source, the deterministic seed when applicable, and the simulated duration. Dispatch and outcome events are published on `/fake_skills/events`.

The same planner and orchestrator path can therefore be exercised with physical results or

with all-success, fail-once, and seeded pseudo-random simulated outcomes. Stateful policies use deterministic call counters and recorded seeds. Successful simulated results represent the corresponding world-state changes, which are reflected in KnowledgeCore so that later plan steps and dialogue turns observe the updated scenario rather than the initial fixture.

Controlled skill-adaptor evidence supports software-level claims about route selection, plan validity, ordered dispatch, typed feedback, replanning, clarification, report coverage, and symbolic postconditions. Deterministic simulation also makes it possible to place a known failure at a selected execution step and observe whether the remaining goal is safely replanned. Physical navigation accuracy, grasp reliability, manipulation success, perception accuracy, NAOqi timing, and human-robot safety require separate embodied evaluation.

4.5 Models and Runtime Configuration

The implementation can run with the NAO adapters or deterministic simulated skill adapters while retaining the same dialogue, grounding, planning, orchestration, and result contracts. Model and backend selection are supplied through runtime configuration, so changing an inference provider does not require a different ROS 2 interface.

The primary evaluated configuration uses `Qwen3-VL-30B-A3B-Instruct-AWQ` for chatbot and planner inference through an OpenAI-compatible vLLM endpoint. The planner uses temperature 0.1 and an 800-token output bound. Additional Qwen3.5 and Ollama-hosted models were exercised over narrower evidence sets and are reported separately in Chapter 6.

Interface compatibility does not imply equivalent semantic behaviour. Model, backend, prompt version, sampling configuration, and token limits are therefore recorded as part of the experimental configuration and compared under the controls defined in Chapter 5.

Chapter 5

Validation Methodology

5.1 Validation Scope

The evaluation tests whether the designed system works from a user request to a correct response or robot action. A case passes only when the request is interpreted correctly, the relevant world information is used, any required plan contains suitable skills in the correct order, the orchestrator executes only the accepted plan, and the final response agrees with the observed outcome.

The unit of analysis is one user request under a fixed software, model, world-state, and skill configuration. The main questionnaire is entered through the ROS `rqt` chat interface, which provides repeatable text input to the same dialogue component used by the integrated system. These runs do not claim to evaluate automatic speech recognition. Robot speech output is checked separately when it is part of the case.

5.2 Validation Levels

Table 5.1 separates the claims that can be supported by language and software evidence from those that require a physical observation.

Table 5.1: Validation levels and supported claims.

Level	Required evidence	Supported claim
Semantic response	Correct dialogue/knowledge route, relevant grounded entities, and an appropriate response or planner request.	The system understood the tested request and used the declared symbolic context.
Software execution	Valid plan, required skills in the correct sequence, typed skill results, and a truthful final report.	The integrated software completed or safely rejected the tested task.
Physical outcome	Direct robot, camera, or sensor observation of the requested effect.	The tested physical behaviour occurred under the stated conditions.

Controlled skill adapters support semantic-response and software-execution claims by exposing the same call and result contracts as the robot-facing skills. Deterministic simulated outcomes provide repeatable success and failure conditions, while physical claims still require direct robot or sensor observation. The validation is organised into the following stages:

1. **Perception and grounding:** checks that simulated or detected people and objects are represented with the correct identifiers, types, properties, and relations. In software-only runs, the people and objects are simulated by inserting controlled scene and KB facts.
2. **Intent and routing:** checks that dialogue and knowledge questions do not request a plan, while action requests do.
3. **Planner:** checks that an admitted action request becomes a valid sequence of available skills with the required arguments.
4. **Skill execution:** checks ordered execution, controlled failures, replanning or clarification, world-state updates, and truthful reporting without requiring hardware.
5. **Full end-to-end test:** checks the complete path from an `rqt` chat request through dialogue, grounding, planning, orchestration, skills, and the final response. Here, *end-to-end* means that all software components of the designed system participate in the same case.

5.3 Acceptance Criteria

Table 5.2: Validation goals, acceptance rules, and evidence sources.

Goal	Pass rule	Evidence source
Route correctness	The observed route equals the declared route. Dialogue and knowledge turns emit no planner request or execution feedback.	Chatbot turn record, planner-request topic, and plan topic.

Table 5.2 continued

Goal	Pass rule	Evidence source
Semantic-decision uniqueness	One user turn produces one validated route before acknowledgment. A dialogue response followed by unintended execution is a failure even when both paths are individually observable.	Turn identifier, route decision, planner request, dialogue response, and speech events.
Plan validity	Every candidate plan admitted for execution passes the Contract 5 schema and the Contract 4 skill-name, argument, ordering, failure-policy, and lineage checks.	Planner decision record, / <i>intents</i> , and orchestrator validation events.
Skill and order coverage	All required subgoals appear, forbidden skills are absent, and the declared action-order constraints hold.	Expected-plan definition and observed plan steps.
Grounding use	Required fixture entities appear in Contract 3 with canonical identifiers and the declared people, object, and location roles.	KB pre-query, / <i>scene/summary</i> , planner request, and grounded-context record.
Target-set and role integrity	The selected, planned, executed, and reported members equal the declared target set. People remain recipients or human targets; locations and supports remain anchors.	Case definition, target selection, admitted plan, execution events, and structured report outcome.
Execution validation	Only admitted steps are dispatched, and every dispatch has a Contract 6 result and Contract 7 feedback with matching lineage.	Skill action events and / <i>planner/execution_feedback</i> .
Safe failure	An injected failure ends in a valid replan, clarification, help request, or truthful terminal failure. A false completion is a failure.	Controlled skill result, plan versions, dialogue act, and final response.
Report coverage	Every selected member with execution evidence appears in the structured outcome with the correct status; the final response must not claim completion when evidence is incomplete.	Contract 6 results, report outcome, report-result event, and final response.

Table 5.2 continued

Goal	Pass rule	Evidence source
State freshness	The pre-case fixture is verified, undeclared earlier entities are absent, and successful material effects are visible in a post-case KB query.	Fixture definition, pre/post /kb/query, and skill world-state updates.

5.4 Full End-to-End Questionnaire

The questionnaire in Table 5.3 contains dialogue, knowledge, atomic-action, composite-action, and route-safety cases. Every dialogue and knowledge case explicitly forbids a planner request. The knowledge cases cover common entity features—name, type, colour, count, and location relation—as well as a KB update that remains outside the robot-action planner.

Table 5.3: Twenty-two-case full end-to-end questionnaire.

Case ID	Exact utterance	Expected route	Required behaviour
dialogue_greeting	“Hey, how are you?”	dialogue	One natural response; no planner request.
dialogue_favorite_movie	“What is your favourite movie?”	dialogue	Natural answer; no planner request.
dialogue_movie_followup	“My favourite movie is Back to the Future!”	dialogue	Contextual acknowledgement; no planner request.
dialogue_weekend_plans	“Do you have any ideas for this weekend?”	dialogue	Advice only; no planner request.
dialogue_speed_of_light	“What is the speed of light?”	dialogue	Informational answer; no planner request.
kb_visible_now_baseline	“What can you see now?”	knowledge	Answer from the declared state; no planner request.
kb_probe_scene_update	“What new objects can you see?”	knowledge	Use the newly inserted scene facts; no planner request.
kb_object_on_table	“Which object is on the table?”	knowledge	Return the correct support relation; no planner request.
kb_cup_name	“What is the cup called?”	knowledge	Return the stored name; no planner request.
kb_multi_object_count	“How many objects can you see?”	knowledge	Count objects without counting people or tables; no planner request.
kb_cup_colour	“What colour is the cup?”	knowledge	Return the stored colour; no planner request.
kb_update_red_cup	“Add a red cup to your knowledge base.”	knowledge	Update symbolic state only; no robot plan is requested.
skill_head_up	“Move your head up.”	execution	Execute the required motion and report the result.
skill_pick_phone_generic	“Pick up the phone.”	execution	Ground one phone before dispatching <code>pick_object</code> .
skill_pick_object_on_table	“Pick up the object on the table.”	execution	Use the support relation to select one object.
skill_look_at_person	“Look at ALEX.”	execution	Keep ALEX as a person and gaze target.

Table 5.3 continued

Case ID	Exact utterance	Expected route	Required behaviour
composite_head_all_directions	“Move your head up, down, left, and right.”	execution	Preserve all requested motions in the correct order.
composite_bring_objects_to_person	“Bring all the objects you can see to ALEX.”	execution	Preserve the object set and the human recipient.
composite_kitchen_cup_to_operator	“Go to the kitchen, find the cup, and bring it to me.”	execution	Preserve the location, object, recipient, and action order.
composite_go_to_person_wave_report	“Go to ALEX, wave, and tell me when you are finished.”	execution	Navigate, perform the greeting motion, and report completion.
composite_walk_every_object_reports	“Visit every object and tell me when you reach each one.”	execution	Visit each object once and report each arrival.
dialogue_non_action_recipient_memory	“Remember that ALEX is the recipient, but do not act yet.”	dialogue	Acknowledge without a planner request or robot action.

5.5 Skill-Execution and Failure Stress Testing

The validation follows the same observable boundaries for robot-facing and simulated skill adapters. Robot skills provide evidence from the physical platform, whereas deterministic adapters reproduce the same result contract while allowing the outcome of an individual skill call to be selected. This makes it possible to stress planning, dispatch, feedback, replanning, and reporting without changing the request or planner-visible skill definition.

Table 5.4 groups the tests by the interaction or execution boundary they exercise. Dialogue and KB checks establish that non-executable turns remain outside the planner. Single-skill and composite cases then test the transition from grounded request to ordered execution. Failure-management cases examine whether a failed step is correlated with the active goal and produces a valid continuation or truthful termination.

Table 5.4: Validation families and their principal acceptance focus.

Validation family	Path exercised	Acceptance focus
Dialogue interaction	Dialogue manager and chatbot response path	One non-executable route, coherent response, and no planner or skill activity.
Knowledge checks	Chatbot, grounded context, and KnowledgeCore	Correct entities, properties, and relations with no robot-action plan.
Single-skill execution	Grounding, planner, orchestrator, and one principal skill	Correct target and arguments, one admitted execution path, typed result, and truthful report.
Composite skill execution	Planner and orchestrator across an ordered multi-step goal	Complete skill coverage, preserved roles and targets, ordered dispatch, postconditions, and final coverage.
Failure management	Skill result, execution feedback, planner state, and dialogue closure	Correct goal/plan lineage, bounded replan or clarification, no obsolete-step continuation, and no false completion.

Failure injection is interpreted by its position within the accepted goal, as summarised in Table 5.5. A failure at the beginning tests whether the initial plan is stopped before later work is dispatched. A middle-stage failure tests whether completed work and goal lineage are preserved while the remaining sequence is revised. An end-stage failure tests final action or reporting closure. The retained evidence contains controlled beginning- and middle-stage skill failures; end-stage evidence is provided by terminal-report checks rather than by a dedicated last-skill failure injection.

Table 5.5: Failure position and required recovery evidence.

Position	Injected or observed condition	Required evidence
Beginning	The first executable or target-confirmation step fails.	The original plan stops, later obsolete steps are not dispatched, and the system replans, clarifies, or terminates truthfully.
Middle	One or more steps succeed before a later skill fails.	Completed work remains correlated with the goal, the plan version changes when replanning occurs, and only valid remaining work is executed.
End	The final action, report, or terminal communication is unavailable or inconsistent.	Completion is withheld until evidence is complete; retry, a bounded final continuation, or truthful terminal failure is produced.

The expected response depends on the controlled outcome. A successful case must complete all required steps and update the represented world state. A recoverable failure may produce a revised valid plan; missing information must produce clarification; and an unrecoverable failure must produce a truthful final message. No case passes if completion is reported after a failed or unexecuted step.

5.6 Metrics

All rates in Table 5.6 range from 0 to 1. For every metric, 1 is the best possible result and 0 is the worst. Cases for which a metric is not applicable are excluded from that metric’s denominator.

Table 5.6: Validation metrics and their interpretation.

Metric	Definition	Interpretation
Route accuracy	Fraction of cases assigned to the expected dialogue, knowledge, or execution path.	1: every route correct; 0: none correct.
Valid-plan rate	Fraction of execution cases that produce a plan accepted by the orchestrator.	1: every plan valid; 0: no plan valid.
Required-skill coverage	Fraction of required actions that appear in the accepted plans.	1: no required action omitted; 0: all omitted.
Step-order accuracy	Fraction of ordered cases in which the required robot actions occur in the requested sequence.	1: every sequence correct; 0: none correct.
Safe-failure rate	Fraction of injected failures that end in a valid replan, clarification, or truthful failure.	1: every failure handled safely; 0: none handled safely.
Target-set exactness	Fraction of target-bearing cases in which the executed objects or people exactly match those requested.	1: every target set exact; 0: none exact.
Role integrity	Fraction of cases in which people remain people, objects remain objects, and locations remain locations.	1: no role errors; 0: every case has a role error.
Report coverage	Fraction of attempted targets or actions represented correctly in the final report.	1: complete reporting; 0: nothing reported correctly.
Postcondition rate	Fraction of required world-state changes confirmed by the post-execution KB query.	1: every change confirmed; 0: none confirmed.
Decision uniqueness	Fraction of turns that produce one route decision without contradictory dialogue and execution behaviour.	1: every turn consistent; 0: every turn contradictory.

Latency is reported separately using the median, interquartile range, 95th percentile, maximum, and timeout count. A case passes only when every applicable acceptance criterion succeeds. Any missing mandatory element, including terminal closure, is reported as a failure. A result is excluded from scoring only when the required observation is unavailable or cannot be correlated reliably.

5.7 Model and Configuration Variability

The ROS 2 interfaces and runtime contracts are model-independent, but the semantic outputs of compatible models are not assumed to be invariant. Two models can receive the same user request, grounded entity set, skill registry, and output schema while selecting different routes, target sets, arguments, or action sequences. Consequently, endpoint compatibility establishes integration compatibility only; it does not transfer task-level qualification from one model to another.

Model comparisons are interpreted only when the case set, KB fixture, skill configuration, prompt version, sampling parameters, token limits, and execution policy are held constant. Results obtained from different configurations are reported as separate evidence scopes rather than pooled into one score.

Within-model variation is evaluated through repeated prompts under the same configuration. Alternative models are screened first on demanding composite and route-consistency cases; failure on those cases is sufficient to reject equivalence for the tested configuration, but not to construct a universal ranking. A replacement model must complete the full qualification before it can be treated as operationally equivalent.

5.8 Research-Question Evidence Map

Table 5.7 links each research question to the evidence used to answer it.

Table 5.7: Research questions and validation evidence.

RQ	Profile	Main metrics	Claim boundary
RQ1	Planner, skill-execution, and full end-to-end tests	Valid-plan rate, skill coverage, and step-order accuracy	Supported task families only.
RQ2	Perception/grounding and full end-to-end tests	Target-set exactness, role integrity, and postcondition rate	Declared simulated or observed world state.
RQ3	Planner and orchestrator tests	Invalid-plan rejection and absence of unsupported skill dispatch	Tested schemas, arguments, and registered skills.
RQ4	Controlled skill failures	Safe-failure rate and truthful reporting	Retained beginning-, middle-, and end-stage evidence.
RQ5	Full end-to-end tests	Route accuracy, decision uniqueness, and report coverage	Implemented component and interface boundaries.

5.9 Threats to Validity

The fixed questionnaire covers only a bounded part of natural language, and repeated development against known prompts may improve performance specifically on those prompts. The robustness set reduces this risk through paraphrase and repetition but does not estimate performance over unrestricted instructions. Controlled people, objects, and relations also simplify the grounding problem compared with an uncontrolled environment.

Language-model output may vary even when the request, KB entities, prompt, and skill registry are unchanged. A single pass can therefore overstate reliability, while comparisons across different models or backend settings can confound model quality with configuration changes. The methodology addresses this by retaining all repetitions, separating configuration tuples, and limiting cross-model conclusions to matched evidence.

Deterministic simulated adapters make it possible to test the same planner, orchestrator, feedback, and reporting logic independently of the robot platform. This reduces platform dependence and supports reuse through equivalent skill adapters. Simulated results do not measure physical navigation, grasping, manipulation, or safety. Physical observations in this thesis are limited to the capabilities exercised on NAO, while the software-level conclusions apply to platforms that provide equivalent skill and result interfaces.

Chapter 6

Results

6.1 Frozen Evidence Scope

The retained evidence contains 108 case records derived from 33 structured runtime traces. Each record identifies the evaluated model and configuration, the purpose of the test, the observed route, execution evidence, final status, and diagnostic attribution. Complete traces remain authoritative for event ordering and wording.

Table 6.1 separates the primary frozen evaluation from application-level comparisons and alternative-model screening. These scopes have different purposes and denominators and are therefore not combined into a single benchmark score. A missing terminal closure is counted as a failure in this chapter.

Table 6.1: Evaluation evidence by purpose and model scope.

Evidence scope	Model scope	Pass	Fail	Insufficient	Interpretation
Primary frozen evaluation	Qwen3-VL	64	8	0	Full integrated qualification; the case without terminal closure is included among the failures.
Application comparisons	Qwen3-VL	21	5	0	Sampling, token-budget, and timeout comparisons under the primary model.
Alternative vLLM screening	Qwen3.5	0	4	2	Demanding composite and route-consistency probes; two observations lacked reliable correlation.
Alternative cloud-model screening	Gemma and Nemotron	0	3	0	One demanding composite probe per model; all failed to provide equivalent capability coverage.
Operational fallback smoke test	Gemma 31B cloud	1	0	0	One dialogue turn established availability, not replacement qualification.

Across these scopes, 86 records passed, 20 did not meet the applicable acceptance criteria, and two could not be correlated sufficiently for scoring. Attribution remains diagnostic: it identifies the narrowest supported cause without changing the case verdict.

6.2 End-to-End Questionnaire Analysis

The final integrated run is evaluated against the 22 exercised requirements in Table 5.3. It contains ordinary dialogue, KB checks, single-skill execution, composite skill execution, and route-safety cases. Table 6.2 reports 20 passes and two failures. The successful cases demonstrate that the required component boundaries can operate coherently, while the failures show that repeated use of those boundaries does not make model-authored routing and role assignment invariant.

Table 6.2: Results for the 22-case end-to-end questionnaire.

Family	Cases	Pass	Fail	Result boundary
Dialogue	5	5	0	Correct dialogue route and no planner request.
Knowledge	7	6	1	Direct entity and relation queries passed; the symbolic update entered the robot-action route.
Single-skill execution	4	4	0	Required target, arguments, execution result, and report were observed.
Composite skill execution	5	4	1	Four composites completed; the kitchen request did not preserve the human-recipient role or reach execution.
Route safety	1	1	0	Recipient memory was acknowledged without action.
Total	22	20	2	Twenty requirements passed and two failed.

The six passing knowledge requirements cover current visibility, scene updates, support relations, object naming, counting, and colour. The red-cup update changed symbolic state, but it produced an execution route, a planner request, and the `kb_add` and `report_result` operations. It therefore fails the requirement that knowledge maintenance remain outside the robot-action planner.

All four single-skill cases passed. Four composite behaviours also passed: ordered head motion, delivery of visible objects to ALEX, navigation to ALEX followed by a wave and completion report, and visiting every object with per-target reports. The recorded wave case uses the label `wave_greet`; the canonical twelve-skill taxonomy represents the same behaviour as a named `perform_motion`. The trace supports the observed behaviour but does not independently validate that registry projection.

The other failure occurred in the request to go to the kitchen and bring the cup to the

operator. Its trace selected the kitchen as `recipient_id`, asked a clarification, and produced no execution feedback. That behaviour was safe because no unsupported action was dispatched, but it did not preserve the required location, object, and human-recipient roles and did not complete the task.

6.3 Environment and Stateful Knowledge Results

The controlled environment profile passed all 11 cases. It covered grouped location delivery, kitchen delivery, a named apple handoff, role separation, and follow-up queries against the declared scene. The stateful KnowledgeCore chain passed all seven cases: additions and revisions were queryable, delivery effects were reflected in subsequent state, a remaining object could be moved, and the final mixed query stayed coherent. These results support RQ2 within the injected symbolic environment; they do not establish detector accuracy.

Table 6.3: Grounding and symbolic-state profiles in the primary frozen evaluation.

Profile	Pass	Cases	Supported observation
Environment	11	11	Declared entities, roles, relations, grouped targets, and follow-up state remained coherent.
Stateful KB stress	7	7	Add, revise, query, delivery post-effects, subsequent movement, and mixed-state querying remained coherent.

6.4 Skill-Execution and Failure-Management Results

The controlled skill-execution set exercised the same planner, orchestrator, feedback, and result contracts used by robot-facing skills. Deterministic adapters supplied repeatable outcomes at individual calls. Seven of the nine integrated cases passed. One omitted a phone from an “every object” target set, and one performed the expected missing-object search and help request but did not produce terminal closure. Both are failures under the stated acceptance criteria.

The retained traces also distinguish failure position within a goal. The primary beginning-stage probe failed the first navigation call, advanced the plan version, preserved all four targets, completed the revised execution, and closed successfully. Additional controlled observations provide the middle- and later-stage evidence: a pick probe first completed object finding, failed `pick_object` at the second step, and then executed a revised sequence containing scan, find, pick, and report; a blocked-delivery probe failed after preparatory finding and scanning, produced replanning, but lacked terminal closure. Because these observations were not part of the frozen primary matrix, they are used qualitatively rather than included in the aggregate counts of Table 6.1. End-stage weakness is therefore evidenced by missing final reporting rather than by a dedicated last-skill failure injection.

Table 6.4: Skill-execution and staged failure-management results.

Evidence family	Failure position	Verdict	Observed behaviour
Integrated controlled-skill set	Mixed controlled outcomes	7 passes, 2 failures	Successful single and composite calls were observable; failures were target omission and absent terminal closure.
Navigation recovery	Beginning	Pass	The first skill failure produced a new plan version, complete target preservation, recovery execution, and closure.
Pick recovery	Middle	Pass	A second-step pick failure produced a revised scan-find-pick-report sequence and completed.
Blocked delivery	Middle/late	Fail	Preparatory work completed and replanning occurred, but the recovery did not produce terminal closure.
Terminal reporting	End	Fail	Intermediate recovery evidence was present, but completion could not be accepted without a final truthful outcome.

6.5 Repeated Robustness and Capability Extremes

Five fixed robustness prompts were repeated three times. Twelve of the fifteen observations passed. Table 6.5 shows that the failures were not tied to one task type: one repetition omitted a member in paraphrased grouped delivery, one omitted an object from a universal object selection, and one multi-turn execution follow-up omitted the required terminal report. Missing-recipient and future-recipient-memory cases passed in all three frozen repetitions, although the separately targeted missing-recipient probe failed as reported above.

Table 6.5: Frozen robustness cases across three repetitions.

Case	R1	R2	R3	Observed instability
Paraphrased grouped delivery	P	P	F	One selected member omitted.
Object-role exclusion	P	P	F	One required object omitted.
Missing recipient	P	P	P	Correct safe behavior in the frozen repetitions.
Future recipient memory	P	P	P	Non-action memory and subsequent role use remained coherent.
Multi-turn execution follow-up	P	F	P	One run lacked the required final report.
Total	5/5	4/5	3/5	12/15 passes.

The seven capability-extreme cases produced five passes and two failures. In both failures, the model-authored plan omitted required capabilities. The registry and orchestrator rejected the incomplete plans before execution and returned truthful failure instead of partially performing a reduced task. This result is important to the safety claim: the stack did not make the model deterministic, but it prevented two incomplete capability-extreme plans from becoming partial robot behavior.

6.6 Model and Backend Evidence

Qwen3-VL completed the full frozen evaluation, robustness repetitions, demanding composite cases, and application comparisons. Alternative models were screened first on demanding composite or route-consistency probes. Failure at this stage was sufficient to reject them as equivalent replacements for the tested configuration, so a full questionnaire was not run for each model. This staged procedure supports qualification decisions but does not provide a universal model ranking.

Table 6.6: Model evidence and qualification boundary.

Model	Screening scope	Observed result	Qualification boundary
QuantTrio/Qwen3-VL-30B-A3B-Instruct-AWQ	Full questionnaire, repeated robustness, staged failures, demanding composites, and application comparisons	Only model to complete the full qualification	Qualified primary; residual variation remains in universal selection, plan coverage, clarification, and closure.
cyankiwi/Qwen3.5-35B-A3B-AWQ-4bit	Two demanding composite probes and four route/acknowledgement probes	Composite probes failed; two route probes failed and two lacked sufficient correlation	Rejected as an equivalent replacement; evidence is too narrow for a general ranking.
gemma4:31b-cloud	One demanding composite probe and one later dialogue smoke turn	Composite probe failed; dialogue smoke turn passed	Operational fallback only; dialogue availability does not establish planning equivalence.
gemma4:cloud	One demanding composite probe	Failed capability coverage	Not qualified for the integrated task scope.
nemotron-3-super:cloud	One demanding composite probe	Failed capability coverage	Not qualified for the integrated task scope.

The comparison therefore separates interface compatibility from demonstrated behaviour. Passing endpoint readiness or one dialogue turn confirms that a model can be connected; it does not transfer the Qwen3-VL planning qualification.

6.7 Answers to the Research Questions

RQ1—Can the architecture produce coherent plans? Yes, for the bounded tested families. The end-to-end questionnaire passed all four single-skill cases and four of five composite cases; the capability-extreme profile passed five of seven. Incomplete capability-extreme plans were rejected rather than partially executed.

RQ2—Can it preserve grounded entities and symbolic state? Yes within the controlled fixtures: the environment profile passed 11/11 and the stateful KB chain passed 7/7. The kitchen recipient-role error and universal-selection omissions show that complete source context does not guarantee correct model-authored selection.

RQ3—Can generated actions be constrained by reviewed interfaces? Yes for the tested schemas and registry. Invalid or incomplete plans did not cross the orchestrator boundary, and the model did not introduce an unregistered executable skill.

RQ4—Can execution failure be handled safely? Beginning-stage navigation and middle-stage pick failures demonstrated valid feedback-driven replanning. Later blocked-delivery and missing-object traces remained less consistent, particularly in terminal closure and user-facing clarification.

RQ5—Does the modular separation improve auditability and reuse? Yes at the software-contract level. The same dialogue, grounding, plan, feedback, and result boundaries were retained across vLLM, Ollama, deterministic simulated adapters, and the NAO adapters. Equivalent behaviour on another robot or under another model still requires its own qualification.

6.8 Result Summary

The frozen evidence supports a qualified but bounded conclusion. Under the primary configuration, 64 of 72 records passed and eight failed the applicable acceptance criteria. The end-to-end questionnaire produced 20 passes and two failures across 22 exercised requirements. The stack handled beginning- and middle-stage skill failures with valid replans, rejected incomplete capability-extreme plans before execution, and preserved coherent symbolic state across the controlled KB chain. Remaining errors were concentrated in model-authored route choice, universal target selection, role preservation, clarification wording, and terminal closure.

The language model remained non-deterministic. The principal result is therefore not invariance of model behavior, but containment: typed contracts, registry validation, deterministic dispatch, lineage-aware feedback, and truthful reporting made variation visible and prevented

the observed incomplete plans from silently becoming unrestricted robot action.

Chapter 7

Discussion

7.1 What the Final Evidence Demonstrates

The strongest result is not that the language model behaved identically across prompts or backends. It did not. The strongest result is that the architecture separated model-generated proposals from deterministic admission and execution sufficiently to observe, reject, or contain the recorded variation.

The primary frozen evaluation produced 64 passes and eight failures across 72 records. The controlled environment and stateful KB profiles passed 11/11 and 7/7 respectively, while the repeated robustness profile passed 12/15 and the capability-extreme profile passed 5/7. The beginning-stage navigation failure and middle-stage pick failure both produced feedback-driven replans with new plan versions; the navigation case also retained complete target coverage and terminal closure. These results support the use of explicit contracts, reviewed skills, typed feedback, and versioned orchestration for bounded language-guided execution.

The failures also define the boundary of that claim. Universal target sets were occasionally incomplete, one multi-turn case omitted closure, one missing-recipient probe did not express the required clarification, and two capability-extreme plans omitted required actions. The architecture prevented the two incomplete plans from executing and did not invent success for unselected members. It did not, however, repair every semantic omission or guarantee a user-facing closure in every recovery case.

7.2 Model-Independent Interfaces and Model-Dependent Behaviour

The runtime interfaces were stable across the primary Qwen3-VL configuration, the Qwen3.5 and cloud-model screening runs, the Gemma fallback smoke test, and the deterministic simulated-skill profiles. Planner requests, compact grounded context, plan admission, execution feedback, and dialogue acts retained the same architectural roles. This is the sense in which

the design is model-agnostic: a model can be changed without redefining the owned interfaces. Behavior was not model-invariant. Qwen3-VL supplied the only complete frozen qualification; Qwen3.5 and the Ollama models had narrower and mostly failing capability-extreme evidence. The current `gemma4:31b-cloud` fallback passed readiness checks and one dialogue smoke turn only. It cannot inherit the Qwen3-VL qualification merely because it uses the same endpoints and schemas.

This distinction is central to the thesis. Model-agnostic integration means stable contracts and a repeatable qualification procedure, not equal output quality from every compatible model. A new model must pass the same questionnaire, robustness repetitions, failure injections, and capability-extreme cases before it can replace the qualified primary.

7.3 Questionnaire-Level Interpretation

The end-to-end questionnaire contains 22 exercised requirements and produced 20 passes and two failures. This result combines simple dialogue, KB checks, single-skill execution, composite execution, and route safety under one integrated evaluation path.

The first failure is a route error. The red-cup update changed KnowledgeCore successfully, but it entered the execution planner and used `kb_add`. Under the final architecture, KB maintenance belongs to the knowledge component and must not be presented as a planner-visible robot skill. The second failure is a semantic role error. The kitchen delivery case selected the kitchen as a recipient, clarified, and stopped without execution. The stop was safe, but the plan path did not preserve the kitchen as a location and the operator as the recipient.

The recorded `wave_greet` case requires a similar boundary. The behavior of navigating, waving, and reporting passed, but the canonical skill taxonomy represents a wave as a named `perform_motion`. The trace supports the behavior, not an independent test of that twelve-skill projection.

7.4 Grounding, Roles, and Symbolic State

Compact symbolic grounding was sufficient for the controlled environment and stateful KB chains. Objects, people, locations, support relations, colours, and post-action state remained queryable across the positive profiles. This confirms the value of providing the planner with a bounded symbolic projection rather than asking it to reconstruct state from dialogue history alone.

The failures show why grounded context must not be confused with grounded selection. The correct entities can be present while the model omits one member from an “all objects” request or assigns a location to the recipient role. The critical interface boundary is therefore the transition from available evidence to authoritative target selection. Validation must

compare requested, selected, planned, executed, and reported members and must preserve role constraints at each boundary.

Knowledge postconditions also matter independently of a successful skill response. A report should be treated as true only when the execution evidence and required state change agree. The 7/7 stateful chain supports this mechanism under the controlled fixtures, while the missing closure case shows that correct intermediate behavior still requires one explicit terminal outcome.

7.5 Planning, Admission, and Replanning

The capability-extreme results distinguish planning quality from execution safety. Five long composites were represented adequately, while two model-authored plans omitted required actions. Both incomplete plans were rejected before dispatch. This is preferable to executing a reduced plan and reporting the original request as complete, but it remains a task-level failure.

The fail-once navigation case provides the clearest beginning-stage recovery evidence. Failure was injected on the first navigation call, correlated feedback was returned, the plan version advanced, the four-target objective was preserved, and the revised execution closed. A separate pick trace exercised the middle of a goal: object finding succeeded, the second-step pick failed, and the revised plan executed scan, find, pick, and report. The architecture can therefore support both complete and remaining-work replanning when failure evidence, objective lineage, and available recovery skills are explicit.

Recovery is less mature for later and terminal failures. A blocked-delivery trace reached the manipulation stage and produced replanning, but did not close; the missing-object case truthfully searched and asked for help but also lacked closure; and the targeted missing-recipient case did not express the expected clarification. The deterministic boundary constrained execution, yet dialogue and closure still depended on model-generated content. Safe non-execution and successful task recovery should therefore remain separate metrics.

7.6 Application Ablations and Non-Determinism

The application ablations did not identify a sampling or token setting that removed semantic variance. Published sampling was slower without a semantic gain, deterministic sampling did not improve the smoke result, and larger token budgets did not repair the capability-coverage failure. Adjusted timeouts improved operational tolerance but did not change the semantic limits.

The A0 discrepancy between automated trajectory checks and manual semantic assessment reinforces the need for layered evidence. All six expected trajectories were present, whereas manual review accepted five of the six responses. Event checks can confirm route and execution presence while semantic review can still identify incomplete wording. Both observations are

therefore retained.

Repeated tests are consequently necessary even when the runtime tuple is frozen. The robustness sequence of 5/5, 4/5, and 3/5 shows that a single passing sample would overstate reliability. The stable architectural claim is that failures remained observable and bounded, not that the same model always generated the same valid plan.

7.7 Controlled Skill Adapters, Physical Claims, and Portability

Deterministic simulated adapters were valuable because they exposed success, failure, re-planning, world-state updates, and reporting under controlled policies while retaining the robot-facing result contract. Their evidence supports software-level claims about admission, dispatch, recovery, and symbolic effects. It does not establish physical navigation accuracy, grasp reliability, manipulation success, or human-robot safety.

The stack is partly platform-independent and partly NAO-specific. The planner request, planner output, execution feedback, dialogue act, and compact grounding contracts can be reused if another platform supplies an equivalent reviewed skill registry and typed adapters. Speech execution, replay motion, posture control, head motion, and look-at behavior in this implementation depend on NAO packages or adapters. Cross-platform reuse is therefore an architectural proposition, not a demonstrated physical result.

ROS 2 and ROS4HRI provide useful modular boundaries but also introduce lifecycle, remapping, dialogue-state, transform, and availability dependencies (Quigley et al. 2009; Macenski et al. 2022; Mohamed et al. 2020). Preflight and trace correlation are part of the experiment because a component that is present but inactive can otherwise resemble a model failure.

7.8 Implications for the Research Questions

RQ1 is supported for the tested task families, with explicit failures on one questionnaire composite and two capability-extreme cases. RQ2 is supported strongly in controlled symbolic profiles, but target selection and role assignment remain model-sensitive. RQ3 is supported by rejection of incomplete plans and by the absence of unregistered dispatch in the retained evidence. RQ4 is supported directly for fail-once navigation and only partially for missing-object and missing-recipient recovery. RQ5 is supported at the interface and auditability level, while equivalent behavior on another robot or model remains unqualified.

The overall interpretation is consequently measured: the architecture converts many language-model failures from silent execution risks into explicit route, selection, plan, clarification, or closure failures. That is a meaningful safety improvement, but it is not a proof of unrestricted autonomy or exhaustive safety.

Chapter 8

Limitations and Future Work

8.1 Current Limitations

The final campaign is scenario-based. Its 108 retained records and the 22-case end-to-end questionnaire provide bounded evidence, not exhaustive coverage of natural language, world states, or failure modes. Repeated development against known prompts may also improve those prompts more than unseen requests.

Only the Qwen3-VL configuration completed the frozen primary qualification. The Qwen3.5 and Ollama observations are heterogeneous historical probes, and the final `gemma4:31b-cloud` selection has only preflight and one dialogue smoke pass. Stable interfaces therefore do not imply equivalent semantic behavior across models.

Residual variance remained visible even with a frozen runtime tuple. The robustness profile passed 12/15 and the capability-extreme profile passed 5/7. Errors included incomplete universal target selection, omitted closure, missing-recipient wording, and incomplete capability-extreme plans. The orchestrator rejected or bounded these cases, but a safe rejection is still a task failure.

The questionnaire exposes two remaining semantic limits. The red-cup update is a route failure under the knowledge/planner separation, and the kitchen request is a role-preservation failure. The recorded wave case uses `wave_greet`, whereas the canonical registry taxonomy represents the behavior through `perform_motion`.

Most execution evidence uses deterministic simulated skill adapters and controlled KB fixtures. This supports software-contract, replan, and symbolic-state conclusions but not physical navigation, grasping, manipulation, perception accuracy, or human safety. Object detection was intentionally disabled in the principal frozen profile, so detector-grounded behavior is outside the main result.

8.2 Future Work

8.2.1 Model Coupling and Qualification

A promising next step is parameter-efficient post-training, such as LoRA, using accepted planner traces and carefully labelled failures. Training examples should target exact role preservation, universal-set coverage, valid JSON, clarification wording, plan completeness, and one truthful terminal closure. Training and evaluation sets must be separated by task template, entity names, and paraphrase family so that improvement is not measured on memorised prompts.

Schema-constrained decoding or grammar-guided generation could reduce malformed outputs, but it will not by itself repair omitted targets or incorrect roles. Any adapted model must therefore repeat the same frozen 22-case questionnaire, 15-run robustness set, failure injections, KB chain, and capability-extreme profile. No replacement should inherit qualification from endpoint compatibility or a smoke test.

8.2.2 Deterministic Semantic Guards

The deterministic boundary can be extended beyond schema and skill validation. Future guards should compare requested and selected cardinalities, enforce object/person/location role types, require every admitted composite capability to appear in the plan, and verify report coverage against attempted targets. A rejected plan should produce a deterministic failure reason that the dialogue layer can express even if the model cannot formulate a clarification.

Terminal closure should become an explicit orchestrator obligation rather than a best-effort model behavior. Similarly, knowledge-only mutations should bypass the robot-action planner by construction, preventing the route failure observed in the red-cup case.

8.2.3 Broader Embodied Validation

The next physical campaign should separately measure navigation, head motion, gaze, speech, and any supported manipulation adapter with exact trial counts and direct observations. A detector-enabled profile should test freshness, identity continuity, ambiguity, and disagreement between perception and authored KB facts. These results should remain separate from deterministic simulated-skill evidence.

Portability should be tested by implementing the same reviewed contracts on a second ROS-compatible platform. Reuse would be demonstrated only if the alternative adapters produce equivalent typed evidence and pass the same software questionnaire without changing planner-facing semantics.

8.2.4 Evidence and Reproducibility

Future campaigns should retain one immutable runtime tuple, machine-readable manifests, raw traces, dataset rows, and a rendered report generated from the same aggregate source. The A0 manual/machine discrepancy and the compact report's source-file-count mismatch show why generated summaries require consistency checks. Automated validation should fail the report build when denominators disagree.

The retained structured traces provide the baseline for future comparison. New experiments should add a separate frozen campaign rather than overwrite the existing evidence.

Chapter 9

Conclusion

This thesis presented the design, integration, and validation of a modular LLM-based interactive architecture, implemented with the NAO robot. The contribution combines dialogue routing, compact symbolic grounding, structured planning, a reviewed skill registry, deterministic orchestration, typed execution feedback, simulated and robot-facing skill adapters, and evidence-based user reporting.

The final evidence package contains 108 retained case records from 33 structured runtime traces. Under the primary frozen configuration, 64 of 72 records passed and eight failed the applicable acceptance criteria. The 22-case end-to-end questionnaire produced 20 passes and two failures. The environment profile passed 11/11, the stateful KB chain passed 7/7, the repeated robustness set passed 12/15, and the capability-extreme set passed 5/7.

The architecture demonstrated a complete feedback-driven recovery in the fail-once navigation case. It also rejected two incomplete capability-extreme plans before execution. These results show that generated plans can be constrained by deterministic interfaces and that model errors need not become unrestricted robot actions. They do not show that every failure can be repaired: universal target selection, role preservation, clarification wording, and terminal closure remained variable.

The model comparison further clarifies the meaning of a model-agnostic stack. The planner and execution contracts remained reusable across backends, but semantic quality did not remain constant. Qwen3-VL supplied the complete frozen qualification. Qwen3.5 and the Ollama models had narrower historical evidence, while the final `gemma4:31b-cloud` fallback passed only preflight and one dialogue smoke turn. A compatible model must complete the same frozen campaign before it can be treated as an equivalent replacement.

Overall, the thesis supports a bounded conclusion: language-model dialogue and planning can participate in a robot architecture when their outputs are grounded, validated against reviewed skills, dispatched deterministically, and checked against typed feedback and symbolic postconditions. These boundaries do not eliminate non-determinism. They make it observable, limit its authority, and provide a reproducible method for deciding whether a particular model and runtime configuration are safe enough for the tested scope.

Appendix A

Full Runtime Contracts

The runtime contracts are described in Section 4.1. This appendix provides a compact reference for the implemented skill interfaces.

A.1 Runtime Skill Inventory

Table A.1 gives the compressed runtime projection used by the planner and orchestrator. The inventory contains the twelve planner-visible robot and interaction skills. Knowledge-base add, revise, and remove operations are not included because they are knowledge-component interfaces rather than robot skills.

Table A.1: Compressed runtime skill inventory and registry values.

Skill and category	Required parameters	Expected effect or evidence	Known failure modes	Executor and status
scan perception	none	refreshed scene evidence and typed scan payload	no_person, objects_only, backend_unavailable, unexpected_payload	orchestrator.scan; first-party
find_object perception	target	object or person evidence is returned	not_found, ambiguous, backend_unavailable	fake.find_object; fake
navigate_to navigation	target	robot arrives at the target location	path_blocked, unknown_location, safety_disabled	fake.navigate_to; fake
walk_to navigation	none	short local walking segment is executed or dry-run	invalid_local_goal, safety_disabled, path_blocked, naoqi_move_failed	orchestrator.walk_to; hybrid
bring_object manipulation	target, recipient	object is delivered to recipient or destination	acquisition_failure, navigation_failure, delivery_blocked, recipient_unavailable	fake.bring_object; fake
place_object manipulation	target, destination	object is released and a support relation is added	destination_unavailable, invalid_support, no_held_object, placement_failed	fake.place_object; fake
pick_object manipulation	target	robot holds the object and updates its support relation	object_unavailable, unreachable, already_held, grasp_failed	fake.pick_object; fake
perform_motion embodiment	object	robot posture or head orientation changes	unsupported_motion, dispatch_failed	orchestrator.perform_motion; first-party
look_at attention	none	robot gaze is redirected or reset	missing_target, dispatch_failed	orchestrator.look_at; first-party
report_result communication	none	user receives a result summary through robot speech	missing_summary, say_dispatch_failed	orchestrator.report_result; first-party
ask_user communication	none	structured clarification request obtains missing user information	missing_prompt, dialogue_relay_unavailable	orchestrator.ask_user; first-party
inspect_area perception	target	scene evidence for the target area is returned	ambiguous, area_unknown, backend_unavailable	fake.inspect_area; fake

The inventory separates each planner-facing skill name from its executor. In the display column, `orchestrator.*` abbreviates `nao_orchestrator.*`, and `fake.*` abbreviates `fake_skills.*`. A greeting wave is represented as a named `perform_motion` action rather than as a separate skill.

Appendix B

Launch Commands

The command below starts the compact planner-enabled simulator profile. Scored validation bundles record the complete launch argument tuple described in Chapter 5.

```
1 ros2 launch nao_chatbot nao_chatbot_sim_demo.launch.py \  
2   start_planner_llm:=true \  
3   chatbot_planner_mode_enabled:=true \  
4   planner_llm_think:=false
```

Appendix C

Validation Log Template

Table C.1 shows the fields used to index one validation case. Machine-readable traces and run manifests remain the authoritative evidence for aggregate results.

Field	Value
Scenario ID	<i>S1–S8</i>
User command	<i>Paste command</i>
Model	<i>Model name</i>
Planner request	<i>Path or pasted payload</i>
Planner output	<i>Path or pasted payload</i>
Execution feed-back	<i>Path or pasted payload</i>
Dialogue act	<i>Path or pasted payload</i>
Outcome	<i>success / failure / recovered / clarification</i>
Notes	<i>Observations</i>

Table C.1: Validation log template.

Appendix D

Final Evaluation Dataset

D.1 Dataset Scope and Attachment

The final dataset is a provenance index over the retained runtime artifacts. It contains 108 case records and 24 fields. The raw JSON files in the evidence archive remain authoritative for complete event order, payloads, and timing. The accompanying machine-readable index is embedded in this PDF as an attachment:

Embedded dataset: `dataset.csv` 

PDF viewers differ in how they display attachments. If the paperclip icon is not interactive, the external file with the matching SHA-256 value in Table D.2 is the same dataset used for the reported aggregates.

D.2 Run Inventory

Table D.1: Machine-derived archive inventory used by the final dataset.

Archive group	Source files	Case records	Purpose
current_v34	12	72	Frozen primary qualification, including two zero-case state snapshots.
ablations	13	26	Sampling, token-budget, timeout, and startup-state evidence.
historical_vllm	3	6	Earlier Qwen3.5 route and capability-extreme evidence.
historical_ollama	3	3	Historical reserve-model capability-extreme probes.
current_ollama_switch	2	1	Model-resolution record and current Gemma smoke result.
Total	33	108	Complete retained index.

D.3 Field Dictionary

The 24 machine-readable fields are grouped as follows:

- **Provenance and identity:** `record_id`, `source_file`, `run_group`, `backend`, `model`, `case_set`, `case_name`, and `category`.
- **Adjudication:** `status`, `expected_outcome`, `reasons`, and `attribution_class`.
- **Context and consistency:** `all_required_context`, `evidence_consistent`, and `route_observed`.
- **Runtime observations:** `planner_request_observed`, `execution_feedback_observed`, `clarification_observed`, `failure_observed`, and `replan_observed`.
- **Event counts:** `fallback_total`, `target_selection_count`, `executed_skill_count`, and `spoken_text_count`.

The source status codes are *pass*, *degraded*, *fail*, and *not scored*. The complete index below abbreviates them as P, D, F, and NS. Run groups are V34, ABL, HVL, HOL, and CUR. Model codes are QVL for Qwen3-VL, Q35 for Qwen3.5, G31 for `gemma4:31b-cloud`, G4 for `gemma4:cloud`, and NEM for `nemotron-3-super:cloud`. Attribution codes are SC for coherent stack success, MV for model/backend variance, HO for harness observability, and SR for stack contract/runtime.

D.4 Evidence Integrity

Table D.2: SHA-256 identifiers for the final supplied evidence files.

File	SHA-256
dataset.csv	be4d7d9b0499557cb9a16ba08cba65d58c011863affcaff74211b3e6c57931ce
nao_ros4hri_model_agnostic_evidence_2026-07-20.zip	9e49b71bd633a8f2005858bbb145dcb5be4917cbf37c2f34c6e6ee05d65b1992
Model_Agnostic_Runtime_Qualification_Report.docx	225c38378bfc175c69c8bb9d23b1dcdb90516473607026d8d99d4d4a8d7667c7

D.5 Complete Case Index

Table D.3 lists every retained record. It is an index, not a replacement for the raw trace artifacts.

Table D.3: Complete index of the 108 retained case records.

Record ID	Group	Case name	Model	Status	Attr.
nao_qvenvl_a0_smoke#1	ABL	simple_dialogue_hey	QVL	P	SC
nao_qvenvl_a0_smoke#2	ABL	kb_visible_now	QVL	P	SC
nao_qvenvl_a0_smoke#3	ABL	kb_injected_object_name	QVL	P	SC
nao_qvenvl_a0_smoke#4	ABL	simple_wave	QVL	P	SC
nao_qvenvl_a0_smoke#5	ABL	composite_head_wave	QVL	P	SC
nao_qvenvl_a0_smoke#6	ABL	reflective_followup	QVL	P	SC
nao_qvenvl_a0max_capability#1	ABL	extreme_all_objects_visit_look_wave_sit	QVL	F	MV
nao_qvenvl_a1r_smoke#1	ABL	simple_dialogue_hey	QVL	P	SC
nao_qvenvl_a1r_smoke#2	ABL	kb_visible_now	QVL	P	SC
nao_qvenvl_a1r_smoke#3	ABL	kb_injected_object_name	QVL	P	SC
nao_qvenvl_a1r_smoke#4	ABL	simple_wave	QVL	P	SC
nao_qvenvl_a1r_smoke#5	ABL	composite_head_wave	QVL	P	SC
nao_qvenvl_a1r_smoke#6	ABL	reflective_followup	QVL	F	MV
nao_qvenvl_a2_smoke#1	ABL	simple_dialogue_hey	QVL	P	SC
nao_qvenvl_a2_smoke#2	ABL	kb_visible_now	QVL	P	SC
nao_qvenvl_a2_smoke#3	ABL	kb_injected_object_name	QVL	P	SC
nao_qvenvl_a2_smoke#4	ABL	simple_wave	QVL	P	SC
nao_qvenvl_a2_smoke#5	ABL	composite_head_wave	QVL	P	SC
nao_qvenvl_a2_smoke#6	ABL	reflective_followup	QVL	F	MV
nao_qvenvl_a3_capability#1	ABL	extreme_all_objects_visit_look_wave_sit	QVL	F	MV
nao_qvenvl_a4_smoke#1	ABL	simple_dialogue_hey	QVL	P	SC
nao_qvenvl_a4_smoke#2	ABL	kb_visible_now	QVL	P	SC
nao_qvenvl_a4_smoke#3	ABL	kb_injected_object_name	QVL	P	SC
nao_qvenvl_a4_smoke#4	ABL	simple_wave	QVL	P	SC
nao_qvenvl_a4_smoke#5	ABL	composite_head_wave	QVL	P	SC
nao_qvenvl_a4_smoke#6	ABL	reflective_followup	QVL	F	MV
nao_ollama_gemma_smoke#1	CUR	simple_dialogue_hey	G31	P	SC
capability_extreme_v34#1	V34	extreme_walk_pick_sit_report	QVL	P	SC
capability_extreme_v34#2	V34	extreme_kneel_under_table_pick_report	QVL	P	SC
capability_extreme_v34#3	V34	extreme_dialogue_inventory	QVL	P	SC
capability_extreme_v34#4	V34	extreme_dialogue_sit_stand_grab_return	QVL	F	MV
capability_extreme_v34#5	V34	extreme_all_objects_visit_look_wave_sit	QVL	F	MV
capability_extreme_v34#6	V34	extreme_pick_place_kneel_report	QVL	P	SC
capability_extreme_v34#7	V34	extreme_unreachable_object_recovery	QVL	P	SC
environment_v33#1	V34	environment_baseline_table_inventory	QVL	P	SC
environment_v33#2	V34	environment_lab_table_inventory	QVL	P	SC
environment_v33#3	V34	environment_lab_sections_inventory	QVL	P	SC
environment_v33#4	V34	environment_lab_sections_delivery	QVL	P	SC
environment_v33#5	V34	environment_kitchen_inventory	QVL	P	SC
environment_v33#6	V34	environment_grouped_location_delivery	QVL	P	SC
environment_v33#7	V34	environment_grouped_location_followup	QVL	P	SC
environment_v33#8	V34	environment_iiia_floor_inventory	QVL	P	SC
environment_v33#9	V34	environment_iiia_kitchen_delivery	QVL	P	SC
environment_v33#10	V34	environment_gold_apple_handoff	QVL	P	SC
environment_v33#11	V34	environment_gold_apple_followup	QVL	P	SC
fail_once_navigation_v34#1	V34	fake_deep_ordered_walk_report	QVL	P	SC
fake_deep_all_success_v34#1	V34	fake_deep_baseline_inventory	QVL	P	SC
fake_deep_all_success_v34#2	V34	fake_deep_ordered_walk_report	QVL	F	MV
fake_deep_all_success_v34#3	V34	fake_deep_grouped_work_table_delivery	QVL	P	SC
fake_deep_all_success_v34#4	V34	fake_deep_missing_object_recovery	QVL	D	MV
fake_deep_all_success_v34#5	V34	fake_deep_missing_recipient_clarification	QVL	P	SC
fake_deep_all_success_v34#6	V34	fake_deep_iiia_kitchen_delivery	QVL	P	SC
fake_deep_all_success_v34#7	V34	fake_deep_iiia_floor_location_followup	QVL	P	SC
fake_deep_all_success_v34#8	V34	fake_deep_gold_apple_multiturn	QVL	P	SC
fake_deep_all_success_v34#9	V34	fake_deep_gold_apple_followup	QVL	P	SC
kb_stress_v33#1	V34	kb_stress_seed_inventory	QVL	P	SC
kb_stress_v33#2	V34	kb_stress_revise_support	QVL	P	SC
kb_stress_v33#3	V34	kb_stress_revised_relation_query	QVL	P	SC
kb_stress_v33#4	V34	kb_stress_grouped_delivery	QVL	P	SC
kb_stress_v33#5	V34	kb_stress_delivery_postcondition	QVL	P	SC
kb_stress_v33#6	V34	kb_stress_move_remaining_object	QVL	P	SC
kb_stress_v33#7	V34	kb_stress_mixed_final_query	QVL	P	SC
main_v33#1	V34	dialogue_greeting	QVL	P	SC
main_v33#2	V34	dialogue_favorite_movie	QVL	P	SC
main_v33#3	V34	dialogue_movie_followup	QVL	P	SC
main_v33#4	V34	dialogue_weekend_plans	QVL	P	SC
main_v33#5	V34	dialogue_speed_of_light	QVL	P	SC
main_v33#6	V34	kb_visible_now_baseline	QVL	P	SC
main_v33#7	V34	kb_probe_scene_update	QVL	P	SC

Continued on next page

Table D.3 continued

Record ID	Group	Case name	Model	Status	Attr.
main_v33#8	V34	kb_object_on_table	QVL	P	SC
main_v33#9	V34	kb_cup_name	QVL	P	SC
main_v33#10	V34	kb_multi_object_count	QVL	P	SC
main_v33#11	V34	skill_head_up	QVL	P	SC
main_v33#12	V34	skill_pick_phone_generic	QVL	P	SC
main_v33#13	V34	skill_pick_object_on_table	QVL	P	SC
main_v33#14	V34	skill_look_at_person	QVL	P	SC
main_v33#15	V34	kb_mutation_add_red_cup	QVL	P	SC
main_v33#16	V34	composite_head_all_directions	QVL	P	SC
main_v33#17	V34	composite_bring_every_object_to_person	QVL	P	SC
main_v33#18	V34	maximal_kitchen_cup_to_operator	QVL	P	SC
main_v33#19	V34	composite_go_to_person_wave_report	QVL	P	SC
main_v33#20	V34	composite_walk_every_object_reports_main	QVL	P	SC
main_v33#21	V34	dialogue_non_action_recipient_memory	QVL	P	SC
missing_recipient_v34#1	V34	robust_missing_recipient	QVL	F	MV
robustness_v34_r1#1	V34	robust_paraphrased_group_delivery	QVL	P	SC
robustness_v34_r1#2	V34	robust_object_role_exclusion	QVL	P	SC
robustness_v34_r1#3	V34	robust_missing_recipient	QVL	P	SC
robustness_v34_r1#4	V34	robust_future_recipient_memory	QVL	P	SC
robustness_v34_r1#5	V34	robust_multiturn_execute_followup	QVL	P	SC
robustness_v34_r2#1	V34	robust_paraphrased_group_delivery	QVL	P	SC
robustness_v34_r2#2	V34	robust_object_role_exclusion	QVL	P	SC
robustness_v34_r2#3	V34	robust_missing_recipient	QVL	P	SC
robustness_v34_r2#4	V34	robust_future_recipient_memory	QVL	P	SC
robustness_v34_r2#5	V34	robust_multiturn_execute_followup	QVL	F	MV
robustness_v34_r3#1	V34	robust_paraphrased_group_delivery	QVL	F	MV
robustness_v34_r3#2	V34	robust_object_role_exclusion	QVL	F	MV
robustness_v34_r3#3	V34	robust_missing_recipient	QVL	P	SC
robustness_v34_r3#4	V34	robust_future_recipient_memory	QVL	P	SC
robustness_v34_r3#5	V34	robust_multiturn_execute_followup	QVL	P	SC
extreme_all_objects_ollama_gemma4#1	HOL	extreme_all_objects_visit_look_wave_sit	G4	F	SR
extreme_all_objects_ollama_gemma4_31b#1	HOL	extreme_all_objects_visit_look_wave_sit	G31	F	MV
extreme_all_objects_ollama_nemotron_super#1	HOL	extreme_all_objects_visit_look_wave_sit	NEM	F	MV
extreme_all_objects_v29_vllm_qwen35#1	HVL	extreme_all_objects_visit_look_wave_sit	Q35	F	MV
extreme_all_objects_v29_vllm_qwen35_	HVL	extreme_all_objects_visit_look_wave_sit	Q35	F	MV
budget2048#1					
route_ack_v29_vllm_qwen35#1	HVL	route_ack_capability_question	Q35	NS	HO
route_ack_v29_vllm_qwen35#2	HVL	route_ack_polite_execution	Q35	F	MV
route_ack_v29_vllm_qwen35#3	HVL	route_ack_polite_followup	Q35	NS	HO
route_ack_v29_vllm_qwen35#4	HVL	route_ack_explicit_execution	Q35	F	MV

Bibliography

- Ahn, Michael, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. (2022). “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances”. In: *arXiv preprint arXiv:2204.01691*.
- Bastianelli, Emanuele, Domenico D. Bloisi, Roberto Capobianco, Guglielmo Gemignani, Luca Iocchi, and Daniele Nardi (2013). “Knowledge representation for robots through human-robot interaction”. In: *arXiv preprint arXiv:1307.7351*.
- Brohan, Anthony, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. (2023). “RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control”. In: *arXiv preprint arXiv:2307.15818*.
- Driess, Danny, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. (2023). “PaLM-E: An Embodied Multimodal Language Model”. In: *arXiv preprint arXiv:2303.03378*.
- Erol, Kutluhan, James Hendler, and Dana S. Nau (1994). “HTN Planning: Complexity and Expressivity”. In: *Proceedings of the Twelfth National Conference on Artificial Intelligence*. AAAI Press, pp. 1123–1128.
- Fong, Terrence, Illah Nourbakhsh, and Kerstin Dautenhahn (2003). “A survey of socially interactive robots”. In: *Robotics and Autonomous Systems* 42.3–4, pp. 143–166.
- Georgievski, Ilche and Marco Aiello (2014). “An Overview of Hierarchical Task Network Planning”. In: *arXiv preprint arXiv:1403.7426*.
- Goodrich, Michael A. and Alan C. Schultz (2007). “Human-robot interaction: a survey”. In: *Foundations and Trends in Human-Computer Interaction* 1.3, pp. 203–275.
- Gouaillier, David, Vincent Hugel, Pierre Blazevic, Chris Kilner, Jérôme Monceaux, Pascal Lafourcade, Brice Marnier, Julien Serre, and Bruno Maisonnier (2008). “The NAO humanoid: a combination of performance and affordability”. In: *arXiv preprint arXiv:0807.3223*.
- Grassi, Lucrezia, Carmine Tommaso Recchiuto, and Antonio Sgorbissa (2021). “Knowledge triggering, extraction and storage via human-robot verbal interaction”. In: *arXiv preprint arXiv:2104.11170*.
- Huang, Wenlong, Pieter Abbeel, Deepak Pathak, and Igor Mordatch (2022a). “Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents”. In: *International Conference on Machine Learning*.

- Huang, Wenlong, Pieter Abbeel, Deepak Pathak, and Igor Mordatch (2022b). “Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents”. In: *arXiv preprint arXiv:2201.07207*.
- Huang, Wenlong, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, et al. (2022c). “Inner Monologue: Embodied Reasoning through Planning with Language Models”. In: *arXiv preprint arXiv:2207.05608*.
- Kim, Moo Jin, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. (2024). “OpenVLA: An Open-Source Vision-Language-Action Model”. In: *arXiv preprint arXiv:2406.09246*.
- Lemaignan, Séverin and contributors (2026). *KnowledgeCore: an RDF-backed knowledge base for robotic applications*. https://github.com/severin-lemaignan/knowledge_core. Open-source software reused by the implementation; accessed July 20, 2026.
- Li, Junnan, Dongxu Li, Silvio Savarese, and Steven Hoi (2023). “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models”. In: *arXiv preprint arXiv:2301.12597*.
- Liang, Jacky, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng (2022). “Code as Policies: Language Model Programs for Embodied Control”. In: *arXiv preprint arXiv:2209.07753*.
- Liu, Bo, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone (2023a). “LLM+P: Empowering Large Language Models with Optimal Planning Proficiency”. In: *arXiv preprint arXiv:2304.11477*.
- Liu, Haotian, Chunyuan Li, Qingyang Wu, and Yong Jae Lee (2023b). “Visual Instruction Tuning”. In: *arXiv preprint arXiv:2304.08485*.
- Macenski, Steve, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall (2022). “Robot Operating System 2: Design, architecture, and uses in the wild”. In: *Science Robotics* 7.66, eabm6074.
- McDermott, Drew, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins (1998). *PDDL: The Planning Domain Definition Language*. Tech. rep. CVC TR-98-003/DCS TR-1165. Yale Center for Computational Vision and Control.
- Mohamed, Youssef and Séverin Lemaignan (2020). “ROS for Human-Robot Interaction”. In: *arXiv preprint arXiv:2012.13944*.
- Nau, Dana S., Tsz-Chiu Au, Okhtay Ilghami, Ugur Kuter, J. William Murdock, Dan Wu, and Fusun Yaman (2003). “SHOP2: An HTN Planning System”. In: *Journal of Artificial Intelligence Research* 20, pp. 379–404.
- Open Robotics (2025). *Robot Operating System 2 Documentation*. Accessed as project background documentation.
- Quigley, Morgan, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng (2009). “ROS: an open-source Robot Operating System”. In: *ICRA Workshop on Open Source Software*.
- ROS4HRI Community (2025). *ROS4HRI: ROS for Human-Robot Interaction*. Accessed as project background documentation.

- Saxena, Ashutosh, Ashesh Jain, Ozan Sener, Aditya Jami, Dipendra K. Misra, and Hema S. Koppula (2014). “RoboBrain: Large-scale knowledge engine for robots”. In: *arXiv preprint arXiv:1412.0691*.
- Shinn, Noah, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao (2023). “Reflexion: Language Agents with Verbal Reinforcement Learning”. In: *arXiv preprint arXiv:2303.11366*.
- Tenorth, Moritz and Michael Beetz (2013). “KnowRob: A knowledge processing infrastructure for cognition-enabled robots”. In: *The International Journal of Robotics Research* 32.5, pp. 566–590.
- (2017). “Representations for robot knowledge in the KnowRob framework”. In: *Artificial Intelligence* 247, pp. 151–169.
- Valmeekam, Karthik, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati (2023). “On the Planning Abilities of Large Language Models: A Critical Investigation”. In: *arXiv preprint arXiv:2305.15771*.
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao (2022). “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *arXiv preprint arXiv:2210.03629*.